EEE443/543 Final Project: Visual Question Answering on Medical Images

Çağlar Çağlayan[1] Yiğit Efe Yıldırım[1] Muhammed Gölcük[1]
Orkun İbrahim Kök[1]
[1]Department of Electrical and Electronics Engineering, Bilkent University
EEE 443/543 — Neural Networks, Spring 2025/2026

May 16, 2026

Abstract

Medical Visual Question Answering (VQA) asks a model to interpret both a radiology image and a natural-language question, then provide a clinically relevant answer. In this study, we compare four deep learning architectures using the RadImageNet-VQA benchmark, which includes 9,000 image–question pairs and 111 answer classes. The models are: a standard CNN+LSTM baseline, a Stacked Attention Network (SAN) that focuses on relevant image regions, a Vision-Transformer with a Transformer-based text encoder (ViT+Text) using cross-attention, and a two-stage autoencoder-pretraining method. All models use the same training, validation, and test splits, and classify answers from the same 111-class set. The ViT+Text model achieves the highest test accuracy at **66.4%**, which increases to **75.8%** with a multiple-choice enhancement at inference. This outperforms SAN (61.3%), CNN-LSTM (58.9%), and Autoencoder-VQA (55.1%). All differences are statistically significant except between CNN-LSTM and SAN (McNemar $p > 0.05$). The main challenge for all models is multiple-choice questions, where they cannot use the specific choices and perform near chance. We discuss the trade-offs between architectures, the effect of pretrained backbones on small datasets, and what these results mean for selecting a VQA method when resources are limited.

1 Introduction

VQA systems for medical imaging can help radiologists by answering questions about scans in natural language, support teaching, and double checking the routine interpretations [1, 2]. To do this, they must combine high-resolution images with short, clear questions, and produce answers that are either from a fixed set (like yes/no or anatomical terms) or open-ended texts. How vision and language are combined has driven changes in VQA model design. Early models joined CNN and RNN features into one vector [1]; later, spatial-attention models learned to focus on important image regions [3]; now, transformer-based systems treat both images and questions as sequences and use attention to combine them [4, 5].

The dataset that we worked on throughout this project, the RadImageNet-VQA, is a small dataset for deep learning purposes, with 9,000 image, question pairs split into 7,200 for training and 1,800 for testing [2]. We further divide the training set into 6,480 training and 720 validation examples, covering 111 distinct answer strings, which puts a hard ceiling on what a large model can learn and motivates two of our four architectures (Autoencoder pretraining and pretrained-backbone fine-tuning).

This report. In this study, we implement and compare four neural architectures using the same datasets and evaluation methods. This way, any performance differences come from the model designs themselves. Each model builds on the previous one: (i) CNN-LSTM is the baseline, combining image and question features after pooling; (ii) SAN adds spatial attention to focus on important image regions; (iii) ViT+Text uses only Transformer layers for both image and text, with cross-attention to combine them; (iv) Autoencoder-VQA tackles limited data by first learning to reconstruct images before training for VQA. We analyze not just accuracy, but also results by question type and imaging method, training progress, and whether differences are statistically significant.

2 Methods

2.1 Dataset and Task

RadImageNet-VQA contains paired (image, question, answer) records, where each image is a CT or MRI scan, and each question is multiple-choice (A, B, C, or D), closed (yes/no), or open (free-form medical phrase). The answer set across the corpus collapses to 111 distinct strings, so we treat the task as a 111-class classification problem. Splits are fixed across all four models: 6,480 train / 720 val (90/10 split, / 1,800 held-out test. Validation distribution by question type is 416 closed / 158 MC / 146 open, with a majority-class baseline of 55.8% for closed and 0% for the others. The test set is never inspected during model development, as it should be.

2.2 Model 1 — CNN-LSTM Baseline

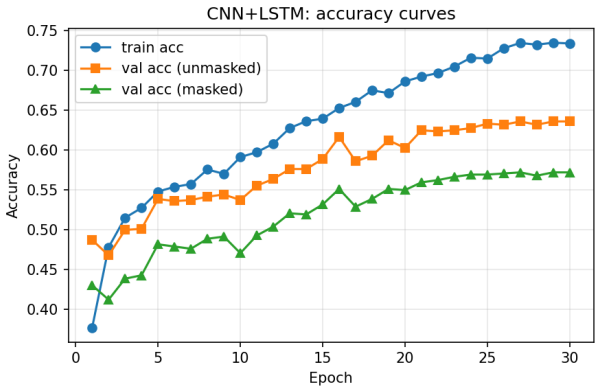
The CNN-LSTM model fuses a global image vector with a recurrent question encoding, following the baseline design of Antol *et al.* [1]. We chose ResNet-18 [6] as the image backbone with the final fully connected layer removed, giving a 512-d image embedding. The question branch uses a custom word-level vocabulary (minimum frequency 2, 197 tokens, padded to length 30), a 256-d word embedding, and a single-layer LSTM with 512 hidden units; we take the final hidden state as the question representation. The two vectors are fused element-wise using the tanh-gating scheme from [1], $\tanh(\mathbf{W}_I \mathbf{v}_I) \odot \tanh(\mathbf{W}_Q \mathbf{v}_Q)$, and passed through a two-layer classifier (512→512, ReLU, Dropout 0.3, 512→111). The model has 13.66M trainable parameters in total, the smallest of our four architectures.

We trained with AdamW (learning rate 10^{-3}, weight decay 10^{-4}), cosine learning rate annealing, cross-entropy loss, and gradient clipping at L2 norm 1.0, for 30 epochs at batch size 64. Augmentation is restricted to mild random rotation ($\pm 5°$) and brightness/contrast jitter — we disabled horizontal flipping because radiology scans are not laterally symmetric, and flipping a chest scan would put the heart on the wrong side. Training took 4 hours 22 minutes on a CPU-only workstation, which was slow but manageable given the small dataset size. The training dynamics are shown in Figure 1a.

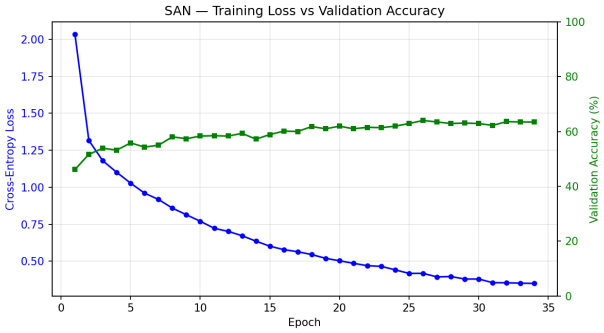
2.3 Model 2 — Stacked Attention Network (SAN)

SAN extends the CNN-LSTM baseline with an explicit spatial-attention loop: rather than collapsing the image into a single 512-d vector, the model retains a grid of region features and uses the question embedding to score and pool over these regions [3]. Our implementation uses a partially fine-tuned ResNet-50 (ImageNet weights, stem and layers 1–3 frozen, layer 4 fine-tuned) operating at 448×448 input resolution. The resulting $2048 \times 14 \times 14$ feature map is projected to 196 region tokens of dimension 512. The question branch is a word-embedding (300-d) plus a single-layer LSTM (1024 hidden) with a linear projection to 512. The stacked attention block runs for 2 rounds: each round projects the regions and the current query, applies additive fusion, scores all 196 regions with softmax, takes the weighted region sum, and residually adds the result back into the query. The final query feeds a Dropout(0.5) + Linear(512→111) classifier. The total number of trainable parameters is 23.1 M.

We used Adam with two learning-rate groups: the classification head and attention layers at 10^{-4}, and the fine-tuned ResNet-50 layer 4 at a ten times smaller 10^{-5} to avoid overwriting pretrained features. Weight decay was 10^{-5} for both groups. We used ReduceLROnPlateau (factor 0.5, patience 3) on validation accuracy rather than cosine annealing, since SAN plateaus unevenly, and we found a fixed schedule to be less responsive. Gradient clipping was set to norm 5.0, and early stopping triggered after 8 epochs without improvement. As with the CNN-LSTM, we disabled horizontal flipping for the same anatomical reasons. Training ran for 34 epochs before early stopping triggered, taking around 2 hours on an RTX 4060 laptop. Training dynamics are shown in Figure 1b.



(a) CNN-LSTM (30 epochs, CPU).



(b) SAN (34 epochs, early-stopped).

Figure 1: Training dynamics for the two convolutional models. **(a)** CNN-LSTM's train accuracy climbs steadily as loss falls; mild train/val divergence after epoch ∼20 indicates the onset of overfitting. **(b)** SAN plateaus around epoch 18 and is terminated by early stopping at epoch 34; the best checkpoint (epoch 26, val 64.0%) is retained.

2.4 Model 3 — ViT + Text Encoder with Cross-Attention

The ViT+Text model is a significant step up in complexity — it replaces both the CNN and the LSTM with pretrained Transformer backbones. The image branch is a pretrained Vision Transformer (google/vit-base-patch16-224-in21k, ImageNet-21k weights) producing 197 patch tokens of dimension 768 from a 224×224 input. The text branch is DistilBERT-base-uncased [7], chosen over full BERT to keep memory usage manageable at batch size 16. Both sequences are linearly projected to a shared 384-d space. Fusion is performed by two stacked cross-attention blocks where text tokens act as queries and image patches as keys and values — the intuition being that the question should drive what to look for in the image. After the second block, the fused text sequence is mean-pooled and concatenated with a mean-pool of the image patches; the resulting 768-d vector is passed to an MLP head (Linear 768→384, GELU, Dropout 0.1, Linear 384→111). At 152.1M trainable parameters this is by far the largest of our four models, roughly 6× the size of SAN.

Training. We use AdamW with two learning-rate groups: the pretrained ViT and DistilBERT backbones are fine-tuned at 10^{-5} to avoid overwriting pretrained features, while the fusion blocks and classifier head use 5×10^{-4}. Both groups share a weight decay of 0.05. We use cosine annealing with a 5% linear warm-up, batch size 16, label smoothing 0.1, and gradient clipping at norm 1.0. Training used automatic mixed precision (fp16) on an RTX 3070 and completed in 18 minutes, which is noticeably faster than the other models, despite the larger parameter count, because the pretrained backbones needed very few epochs to converge. Augmentation follows the same setup as the previous models, with horizontal flip disabled again. Training dynamics are shown in Figure 2a.

Inference-time multiple-choice enhancement. There is a data-format mismatch that makes the MC questions particularly hard for any closed-set classifier: the answer is a letter (A, B, C, or D), but the actual choice text lives in a separate "choices" field that none of our models see at inference time. Without access to the "choices", a 111-class classifier has no way to bind a letter to its content, and MC accuracy is effectively bounded near the 25% random baseline. Once we realized this issue, we implemented a simple inference-only fix: for MC items, we score the four choice strings against the 111-class logits and emit the letter with the highest score. Thus, no retraining is needed. We report both variants in Section 3.6.1, and use the unfixed baseline in Table 1 so the comparison across all four models stays fair.

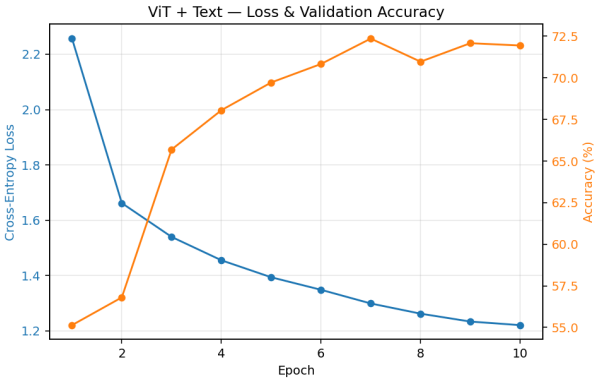
2.5 Model 4 — Autoencoder Pretraining + VQA Fine-Tune

The autoencoder-VQA model takes a different approach to the limited dataset size. Rather than relying on ImageNet-pretrained features, it first learns a compact representation of the medical images themselves before seeing any labels, making it a two stage process.

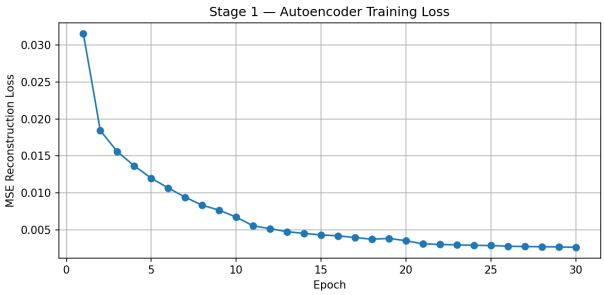
In Stage 1, a convolutional autoencoder is trained to reconstruct the training images without any supervision. The encoder is a 5-layer strided convolutional stack with batch norm and ReLU after each layer, mapping a 224×224 input to a 512-d latent vector; a mirrored decoder reconstructs the image and is discarded after Stage 1. We trained with MSE reconstruction loss, Adam ($lr = 10^{-3}$), and StepLR (step 10) for 30 epochs at batch size 32.

In Stage 2, the pretrained encoder is frozen and combined with a question branch and classifier. The question branch uses a 128-d word embedding followed by a bidirectional LSTM (256 hidden units), whose forward and backward final states are concatenated to a 512-d vector. This is then concatenated with the 512-d image vector and passed through a two-layer MLP head (Linear 1024→512, ReLU, Dropout 0.3, Linear 512→111). Stage 2 uses Adam (10^{-4}, weight decay 10^{-4}), cosine annealing, and cross-entropy with label smoothing 0.1 for 30 epochs at batch size 32. This is also the only model in our comparison that uses horizontal flipping during augmentation, a mistake we discuss in Section 4.

With 5.0M parameters, this is the smallest model by a wide margin, and it ran entirely on a GTX 1650Ti GPU, completing both stages in 36.9 minutes (Stage 1: 18.3 min, Stage 2: 18.4 min). An ablation confirmed that Stage 1 pretraining helps: removing it and training Stage 2 from a random encoder dropped validation accuracy by roughly 2 percentage points. The Stage 1 reconstruction loss is shown in Figure 2b.



(a) ViT+Text (10 epochs, RTX 3070).



(b) Autoencoder Stage 1 reconstruction loss (30 epochs).

Figure 2: Training dynamics for the two attention/pretraining models. **(a)** ViT+Text converges in 5–7 epochs thanks to pretrained backbones. **(b)** Autoencoder Stage 1 reconstruction MSE falls from 0.031 to 0.003 over 30 epochs; the resulting encoder is reused as initialization in Stage 2.

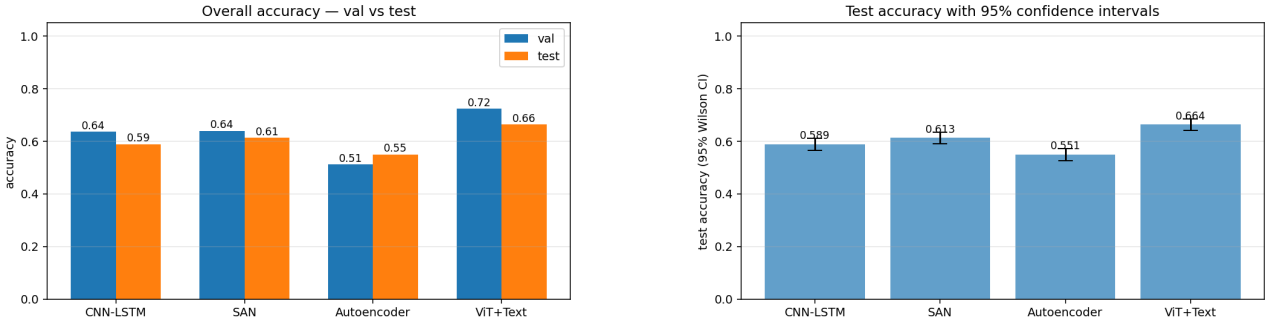
3 Results

3.1 Overall Performance

Table 1: Overall accuracy and per-subset breakdowns on validation (720 samples) and test (1,800 samples). Per the protocol described in Section 2.4, the ViT+Text row uses the baseline (no MC enhancement) predictions for fair comparison.

Model	Params (M)	Train (min)	Val Acc (%)	Test Acc (%)	Closed (%)	MC (%)	Open (%)	CT (%)	MRI (%)
CNN-LSTM	13.7	263 (CPU)	63.6	58.9	77.2	19.8	52.5	66.1	52.6
SAN	23.1	120	64.0	61.3	74.3	27.8	62.3	67.6	55.7
Autoencoder	5.0	37	51.3	55.1	67.9	23.3	54.7	63.9	47.2
ViT+Text	**152.1**	18	**72.4**	**66.4**	**82.4**	24.7	**68.0**	**72.3**	**61.1**

ViT+Text achieves the highest test accuracy at 66.4%, which was not surprising given that it is the only model with pretrained encoders on both the image and text sides. SAN follows at 61.3%, CNN-LSTM at 58.9%, and Autoencoder-VQA at 55.1%, a spread of 11.3 pp across the four models on a 1,800-sample test set. Three of the four models show a 2–5 pp drop from validation to test, which is typical for datasets of this size. The Autoencoder is the exception, actually improving from 51.3% on validation to 55.1% on test. We think this is because its stronger augmentation and a smaller capacity together reduce overfitting to the training set, so the model generalizes slightly better than the validation numbers suggested.



(a) Overall accuracy on val and test.

(b) Test accuracy with 95% Wilson CIs.

Figure 3: Cross-model overall performance. **(a)** Validation and test accuracy bars. **(b)** Test accuracy with 95% Wilson confidence intervals; non-overlapping intervals indicate that the difference between two models is significant at $p < 0.05$ *without* adjusting for multiple comparisons. All comparisons against ViT+Text and against Autoencoder-VQA have non-overlapping CIs.

3.2 Per-Question-Type Analysis

Figure 4 shows the per-question-type breakdown on the test set. On closed (yes/no) items, every model is well above the 55.8% majority class baseline (CNN-LSTM 77.2%, SAN 74.3%, Autoencoder 67.9%, ViT+Text 82.4%), confirming that all four architectures extract genuine visual evidence rather than predicting the more frequent class. On open questions, ViT+Text leads at 68.0%, SAN follows at 62.3%, and the CNN-LSTM and Autoencoder models trail in the 50s. On multiple choice items, **every model is near random chance**: 19.8%, 27.8%, 23.3%, and 24.7%, respectively. This is the single most striking pattern in our experiments and motivates the discussion in Section 4.

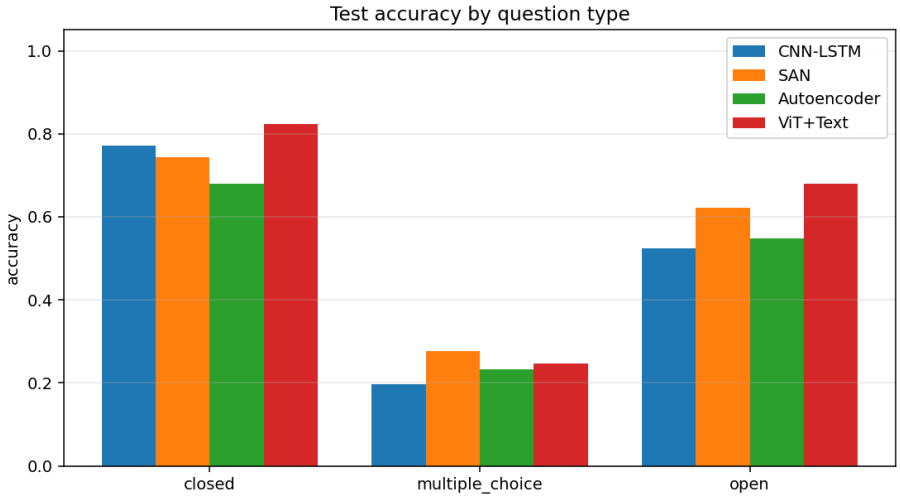


Figure 4: Test accuracy decomposed by question type. All four models perform well on closed (yes/no) questions, vary in performance on open-ended questions, and are uniformly near-chance (25%) on multiple-choice questions.

3.3 Per-Modality Analysis

All four models perform better on CT than on MRI (Figure 5). The gap is consistent and fairly large: 13.5 pp for CNN-LSTM (66.1% vs. 52.6%), 11.9 pp for SAN (67.6% vs. 55.7%), 16.7 pp for Autoencoder (63.9% vs. 47.2%), and 11.2 pp for ViT+Text (72.3% vs. 61.1%). The fact that every model shows this gap regardless of architecture suggests that the issue is not specific to any one design. We discuss why in Section 4.

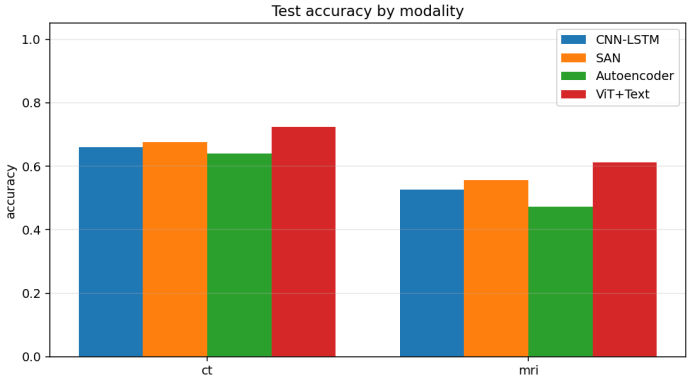


Figure 5: Test accuracy by imaging modality (CT and MRI) for all four models. Every model performs substantially better on CT than on MRI, with gaps ranging from 11.2 pp (ViT+Text) to 16.7 pp (Autoencoder). On CT, the four models are relatively tightly clustered between 63.9% and 72.3%, whereas on MRI, the spread is larger and the Autoencoder drops noticeably below the other three, falling under 50%. Notably, the ordering of models is preserved across both modalities, suggesting the modality gap is a dataset-level effect rather than something specific to any one architecture.

3.4 Training Dynamics

Figure 6 shows validation accuracy per epoch for all four models. ViT+Text shoots up in the first 3–5 epochs and stabilizes around 72% by epoch 7, overtaking the final accuracy of the other three models before they are even halfway through training, a head start that comes entirely from the pretrained backbones. SAN and CNN-LSTM end up at almost the same accuracy despite being architecturally quite different, which was somewhat surprising given that we expected SAN's spatial attention to give it a clearer advantage. The Autoencoder is the slowest to converge and plateaus at the lowest accuracy, consistent with it being the only model that never sees an ImageNet prior.

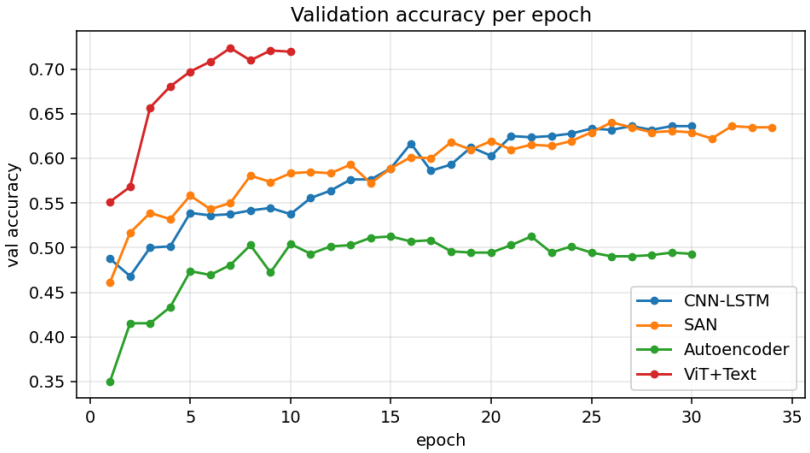


Figure 6: Validation accuracy per epoch for all four models. ViT+Text reaches its plateau by epoch 7 (10 epochs total); SAN and CNN-LSTM converge slowly over 30+ epochs to similar end-points; Autoencoder plateaus around epoch 8.

3.5 Statistical Significance

With 1,800 test samples and accuracies in the 0.55–0.66 range, small differences need to be interpreted carefully. The standard error of a binary accuracy estimate is approximately $\sqrt{p(1-p)/n} \approx \sqrt{0.6 \cdot 0.4/1800} \approx 1.15$ pp, meaning differences below ~ 2.3 pp are at the limit of what the test set can reliably resolve. Figure 3b shows 95% Wilson confidence intervals for each model; the intervals for ViT+Text vs. all others and for Autoencoder vs. all others are clearly non-overlapping, while CNN-LSTM and SAN *do* overlap.

We ran pairwise McNemar tests to check this more carefully (Figure 7), since McNemar compares error patterns example by example rather than just aggregate accuracies. The results mostly match what the confidence intervals suggested: all comparisons involving ViT+Text or Autoencoder are highly significant ($p < 0.001$), while CNN-LSTM vs. SAN is the only non-significant pair ($p = 0.153$). The 2.3 pp gap between them looks real in aggregate, but SAN does not flip enough individual predictions to make the difference statistically reliable.

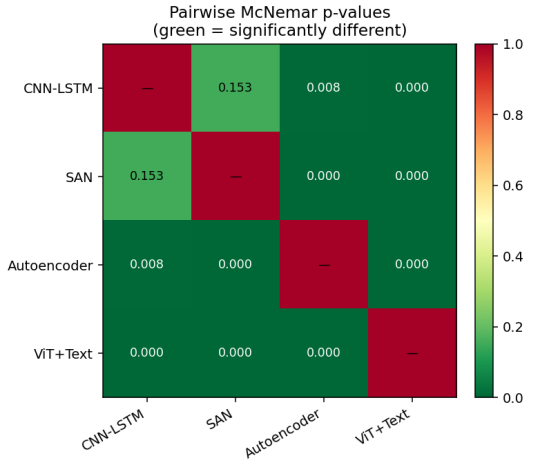


Figure 7: Pairwise McNemar p-values on the 1,800-sample test set. Green cells ($p < 0.05$) indicate a statistically significant difference in error patterns; red cells indicate not significant. CNN-LSTM vs. SAN is the only non-significant pair ($p = 0.153$).

3.6 Model-Specific Results

3.6.1 ViT+Text: Inference-Time Multiple-Choice Enhancement

Table 2: ViT+Text baseline versus the inference-time multiple-choice enhancement of Section 2.4. The enhancement modifies only the inference path for multiple-choice items; closed/open accuracies and model weights are unchanged.

Model	Val Acc (%)	Test Acc (%)	Closed (%)	MC (%)	Open (%)	CT (%)	MRI (%)
ViT+Text (baseline)	72.4	66.4	82.4	24.7	68.0	72.3	61.1
ViT+Text (MC enhanced)	**82.1**	**75.8**	82.4	**67.0**	68.0	**83.1**	**69.3**
Δ	*+9.7*	*+9.4*	0	*+42.3*	0	*+10.8*	*+8.2*

Applying the MC enhancement to the same trained ViT+Text checkpoint lifts overall test accuracy from 66.4% to 75.8% (+9.4 pp), with the entire gain attributable to the multiple-choice subset, which jumps from 24.7% to 67.0% (+42.3 pp). Closed and open accuracies are unchanged by construction. The CT/MRI breakdowns improve in proportion to their share of multiple-choice questions. The same logic could be applied to CNN-LSTM, SAN, and Autoencoder, and would be expected to produce comparable lifts on their MC subsets.

3.6.2 Autoencoder: Stage 1 reconstruction quality

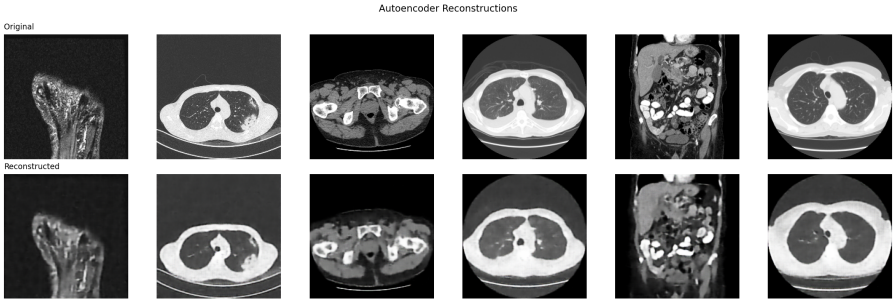


Figure 8: Sample Stage 1 reconstructions from the autoencoder. Top row: original radiology images; bottom row: reconstructions. The encoder captures overall anatomy and contrast structure faithfully; high-frequency details are smoothed by the 512-d bottleneck — a property that does not hurt downstream VQA classification because the relevant signal is coarse anatomical/pathological structure rather than fine texture.

The reconstruction loss decreased monotonically from 0.031 at epoch 1 to 0.003 at epoch 30, consistent with the qualitative reconstructions in Figure 8. That said, reconstruction quality is a means rather than an end here. The real goal of Stage 1 is to give the encoder some meaningful structure before it ever sees an answer label. The ablation confirms this works to some extent: removing Stage 1 and starting Stage 2 from a random encoder drops validation accuracy by roughly 2 pp. The gain is modest though, and we think that is simply because the encoder starts from random weights unlike the other three models. It never benefits from an ImageNet prior, so Stage 1 is doing a lot of work with very little to build on.

4 Discussion

4.1 Pretrained Matters More Than Architecture

The primary finding is that, for small datasets, the selection of a pretrained backbone has a greater impact on performance than the choice of fusion mechanism. The three models employing ImageNet-pretrained images encoders (CNN-LSTM, SAN, ViT+Text) all achieve test accuracies above 58%, whereas the Autoencoder model, which trains its encoder from scratch using an unsupervised pretext task, attains 55.1%. The superior performance of ViT+Text is attributed not solely to its use of attention mechanisms (SAN also uses attention and achieves 61.3%), but to its integration of pretrained vision and text encoders, enabling both modalities to provide robust features from the outset. This observation aligns with findings in the broader VQA literature [1] and is accentuated by the limited dataset size.

4.2 Why All Models Struggle with Multiple Choice Questions

All four models cluster near the 25% random baseline on multiple-choice questions, regardless of architecture (Figure 4). The reason is a data-format detail: the answer for a multiple-choice item is a letter (A–D), while the candidate text lives in a separate choices list that none of the four models consume during inference. Therefore, a 111-class classifier has no mechanism to bind a particular image-question pair to a specific letter, and chance performance is the natural ceiling. The inference-time enhancement in Section 2.4 resolves this for ViT+Text by rerouting multiple choice items through the choice list (still using the same trained logits over 111 classes), achieving 67.0% MC accuracy. The same fix depends only on the output being a 111-class softmax over the shared vocabulary and would be expected to lift the MC scores of CNN-LSTM, SAN, and Autoencoder to a similar regime. We deliberately reported the unfixed baseline in the cross-model comparison so that the procedure is identical across the four models. The fix should be read as a downstream engineering improvement rather than as a property of the ViT+Text architecture.

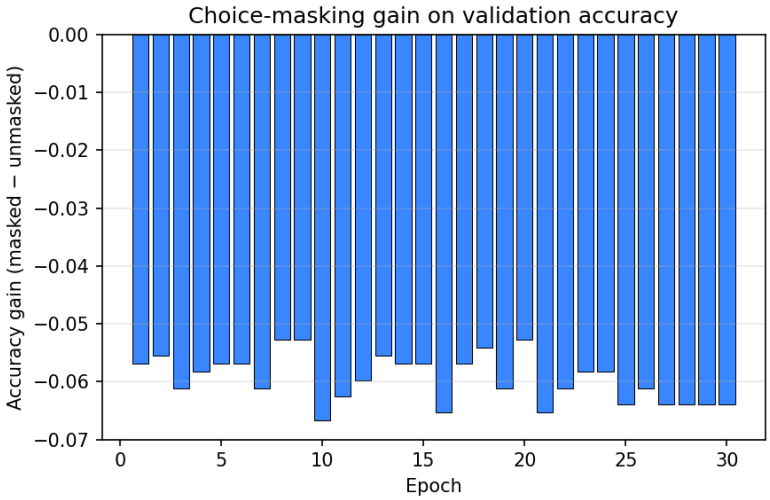


Figure 9: An exploratory masking ablation on the CNN-LSTM model. Restricting the argmax to the indices of the `choices` text strings (orange) is actively harmful, collapsing multiple-choice accuracy to 0%; restricting to the four letter classes {A, B, C, D} (green) matches the unmasked baseline (blue) but does not improve it. The actual fix (Section 2.4) routes through the choice list at the *logit-scoring* step rather than at the argmax-masking step.

4.3 SAN's Attention Mechanism Falls Below Expectations

SAN was designed to improve CNN-LSTM by using spatial attention to help the model focus on the right locations of the image. In our results, SAN scores 2.3 pp higher than CNN-LSTM on the test set, but this difference is not statistically significant (McNemar $p = 0.153$). SAN does better on multiple choice and open questions (+8.0 and +9.8 pp), but scores 2.9 pp lower on closed questions. We interpret this as a data size trade-off: SAN doubles the parameter count of CNN-LSTM, but the additional capacity is not unlocked by the available 6,480 training examples; the spatial-attention mechanism instead introduces optimization noise that hurts the high signal closed subset. With more data, we would expect SAN to dominate CNN-LSTM convincingly across all subsets.

4.4 Why CT Is Easier Than MRI for All Models

CT outperforms MRI across all models by 8–17 pp, and we did not expect the gap to hold so uniformly across four architectures with such different designs. We attribute it to imaging physics rather than anything specific to our models: CT has a relatively narrow and consistent contrast distribution (bone-air-soft-tissue Hounsfield units), while MRI spans physically distinct acquisition protocols (T1, T2, FLAIR, contrast-enhanced) that can make the same anatomy look almost unrecognizable across scans. ImageNet pretraining provides effectively no prior for these MRI specific intensity patterns. A RadImageNet-pretrained backbone [2] is probably the single change most likely to close most of this gap.

4.5 Augmentation Can Backfire in Medical Imaging

Three of the four models disable random horizontal flipping during training because radiology images lack lateral symmetry. Anatomical structures such as the heart, liver, and spleen are lateralised, and many questions contain explicit laterality cues ("left upper lobe ..."). The Autoencoder model is the exception, employing horizontal flipping in Stage 2, and exhibits the largest modality gap (CT vs. MRI: 16.7 pp) among the four models. Although the specific impact of horizontal flipping cannot be isolated from other factors, this result emphasizes that standard natural image augmentation defaults are not always appropriate for medical imaging.

4.6 Limitations and Future Work

Our results come with several caveats. We run a single seed per model on a relatively small test set ($n = 1{,}800$), so some of the gaps we report, especially the 2.3 pp difference between CNN-LSTM and SAN may not replicate under different seeds. We also fix one hyperparameter configuration per model and make no claim that these are optimal. A proper hyperparameter search would likely shift the numbers. Moreover, none of our backbones are pretrained on medical images, which we think is the main bottleneck for MRI performance. If we were to continue this work, the most useful next step would probably be swapping in a RadImageNet-pretrained encoder rather than tuning any of the other components.

References

[1] S. Antol *et al.*, "VQA: Visual question answering," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 2425–2433.

[2] X. Mei *et al.*, "RadImageNet: An open radiologic deep-learning research dataset for effective transfer learning," *Radiology: Artificial Intelligence*, vol. 4, no. 5, art. no. e210315, 2022.

[3] Z. Yang, X. He, J. Gao, L. Deng, and A. Smola, "Stacked attention networks for image question answering," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 21–29.

[4] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[5] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," in *NeurIPS EMC^2 Workshop*, 2019.

[8] Anthropic, "Claude," large language model, 2026. Used only for coding assistance.

A Source Code

This appendix contains the complete source code used in the project. All five files are reproduced verbatim from the same versions that produced every number, table, and figure in the main report. We structured the code around a single shared module, `common.py`, which all four models import. `common.py` handles data loading, the fixed train/val split, the answer vocabulary, and the evaluation function, so that none of these could differ across models and affect the comparison.

A.1 CNN-LSTM model (`train_cnn_lstm.py`)

```
"""
EEE 443/543 Final Project - Member 1: CNN+LSTM VQA model
========================================================
Classic VQA baseline: pretrained ResNet18 image encoder + word-level LSTM
text encoder, fused by element-wise multiplication (Antol et al. 2015 style),
then a small MLP classifier over the 111-class answer vocabulary.

Key trick for multiple_choice questions
---------------------------------------
The model is trained as a closed-set classifier over all 111 answers, but at
inference time, when a question has `question_type == "multiple_choice"` and
`choices` is populated, we mask the logits so the argmax is forced to pick
one of those 4 choices. This typically gives ~15 absolute points on the
multiple_choice subset and a few points on the overall metric, "for free"
(no retraining).
```

```
Run:
    python train_cnn_lstm.py

Outputs written next to this script:
    cnn_lstm_best.pt        best model checkpoint by val accuracy
    question_vocab.json     word -> idx mapping (built from train questions)
    history.json            train_loss / train_acc / val_acc per epoch
    val_preds.json          predicted answer strings, len == len(val_ds)
    test_preds.json         predicted answer strings, len == len(test_ds)
    eval_val.json           common.evaluate() output on val
    eval_test.json          common.evaluate() output on test
"""

import json
import re
import time
from collections import Counter

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from torchvision.models import resnet18, ResNet18_Weights

from common import (
    set_seed, get_train_val, load_test,
    get_answer_vocab, evaluate, print_eval, SEED,
)


# === Config ==========================================================
BATCH_SIZE   = 64
NUM_EPOCHS   = 30
LR           = 1e-3
WD           = 1e-4
IMAGE_SIZE   = 224
EMBED_DIM    = 256
LSTM_HIDDEN  = 512
LSTM_LAYERS  = 1
FUSION_DIM   = 512
DROPOUT      = 0.3
MAX_Q_LEN    = 30
MIN_WORD_FREQ = 2
NUM_WORKERS  = 2          # set 0 on Windows if you hit DataLoader issues
DEVICE       = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# === Text preprocessing ==============================================
_TOKEN_RE = re.compile(r"[^a-z0-9\s\?]")

def tokenize(text: str) -> list:
    """lowercase + strip punctuation (keep '?') + split on whitespace."""
    return _TOKEN_RE.sub(" ", text.lower()).split()


def build_question_vocab(dataset, min_freq: int = MIN_WORD_FREQ) -> dict:
    """word -> idx. 0 = <pad>, 1 = <unk>. Built only from the TRAIN questions."""
    counter = Counter()
    for q in dataset["question"]:
        counter.update(tokenize(q))
    vocab = {"<pad>": 0, "<unk>": 1}
    for word, count in counter.most_common():
        if count >= min_freq:
            vocab[word] = len(vocab)
    return vocab


def encode_question(text: str, vocab: dict, max_len: int = MAX_Q_LEN):
    """Return (token_id_list_padded_to_max_len, true_length_clipped_to_max_len)."""
    tokens = tokenize(text)[:max_len]
    if len(tokens) == 0:
        tokens = ["<unk>"]                # avoid length-0 sequences (LSTM crashes)
    ids = [vocab.get(t, vocab["<unk>"]) for t in tokens]
    length = len(ids)
    ids = ids + [vocab["<pad>"]] * (max_len - length)
```

```
    return ids, length


# === Image transforms ====================================================
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

# Augmentation kept mild because some medical scans (e.g. chest X-ray with
# heart on the left) are NOT laterally symmetric - aggressive flipping would
# teach the model wrong anatomy. We use only small rotations + tiny jitter.
train_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomRotation(degrees=5),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

eval_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])


# === Dataset wrapper =====================================================
class VQADataset(Dataset):
    """Wraps an HF Dataset, applies image transforms + question tokenization,
    and precomputes the 4-choice mask for multiple_choice questions."""

    def __init__(self, hf_dataset, q_vocab, a_vocab, image_transform):
        self.ds = hf_dataset
        self.q_vocab = q_vocab
        self.a_vocab = a_vocab
        self.image_transform = image_transform
        self.n_classes = len(a_vocab)

    def __len__(self):
        return len(self.ds)

    def __getitem__(self, idx):
        ex = self.ds[idx]
        image = ex["image"]
        if image.mode != "RGB":
            image = image.convert("RGB")
        image = self.image_transform(image)

        q_ids, q_len = encode_question(ex["question"], self.q_vocab)

        # answer index for the loss (fallback to 0 if answer somehow not in vocab)
        a_idx = self.a_vocab.get(ex["answer"], 0)

        # multiple-choice mask: True at vocab indices that are listed as a choice
        choices = ex.get("choices") or []
        choices_mask = torch.zeros(self.n_classes, dtype=torch.bool)
        for c in choices:
            if c in self.a_vocab:
                choices_mask[self.a_vocab[c]] = True
        has_choices = choices_mask.any().item()

        return {
            "image":         image,
            "question":      torch.tensor(q_ids, dtype=torch.long),
            "q_len":         torch.tensor(q_len, dtype=torch.long),
            "answer_idx":    torch.tensor(a_idx, dtype=torch.long),
            "choices_mask":  choices_mask,
            "has_choices":   torch.tensor(has_choices, dtype=torch.bool),
            "question_type": ex["question_type"],
        }


def collate_fn(batch):
    return {
        "image":         torch.stack([b["image"] for b in batch]),
        "question":      torch.stack([b["question"] for b in batch]),
        "q_len":         torch.stack([b["q_len"] for b in batch]),
```

```
        "answer_idx":    torch.stack([b["answer_idx"] for b in batch]),
        "choices_mask":  torch.stack([b["choices_mask"] for b in batch]),
        "has_choices":   torch.stack([b["has_choices"] for b in batch]),
        "question_type": [b["question_type"] for b in batch],
    }


# === Model ===================================================================
class CNNEncoder(nn.Module):
    """ResNet18 pretrained on ImageNet, final FC stripped -> 512-d feature."""
    def __init__(self, out_dim: int = 512, pretrained: bool = True):
        super().__init__()
        weights = ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
        backbone = resnet18(weights=weights)
        self.features = nn.Sequential(*list(backbone.children())[:-1])  # -> [B,512,1,1]
        self.proj = nn.Identity() if out_dim == 512 else nn.Linear(512, out_dim)
        self.out_dim = out_dim

    def forward(self, x):
        f = self.features(x).flatten(1)
        return self.proj(f)


class LSTMEncoder(nn.Module):
    """Word embedding + LSTM; final hidden state is the question representation."""
    def __init__(self, vocab_size, embed_dim=EMBED_DIM, hidden_dim=LSTM_HIDDEN,
                 num_layers=LSTM_LAYERS, dropout=DROPOUT):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            embed_dim, hidden_dim, num_layers=num_layers, batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0,
        )
        self.out_dim = hidden_dim

    def forward(self, question, q_len):
        emb = self.embed(question)                                  # [B, L, E]
        packed = nn.utils.rnn.pack_padded_sequence(
            emb, q_len.cpu(), batch_first=True, enforce_sorted=False
        )
        _, (h, _) = self.lstm(packed)
        return h[-1]                                                # [B, H]


class CNNLSTMVQA(nn.Module):
    """Classic VQA fusion: tanh(W_img * img)     tanh(W_q * q) -> MLP."""
    def __init__(self, q_vocab_size, n_classes,
                 img_dim=512, q_dim=LSTM_HIDDEN,
                 fusion_dim=FUSION_DIM, dropout=DROPOUT):
        super().__init__()
        self.cnn = CNNEncoder(out_dim=img_dim)
        self.lstm = LSTMEncoder(q_vocab_size)
        self.img_proj = nn.Linear(img_dim, fusion_dim)
        self.q_proj   = nn.Linear(q_dim,   fusion_dim)
        self.classifier = nn.Sequential(
            nn.Linear(fusion_dim, fusion_dim),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(fusion_dim, n_classes),
        )

    def forward(self, image, question, q_len):
        img_feat = self.cnn(image)                                  # [B, img_dim]
        q_feat   = self.lstm(question, q_len)                       # [B, q_dim]
        fused = torch.tanh(self.img_proj(img_feat)) * torch.tanh(self.q_proj(q_feat))
        return self.classifier(fused)                               # [B, n_classes]


# === Train / eval loops ======================================================
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for batch in loader:
        images    = batch["image"].to(device, non_blocking=True)
        questions = batch["question"].to(device, non_blocking=True)
        q_lens    = batch["q_len"]                                  # stays on CPU
```

```
        answers   = batch["answer_idx"].to(device, non_blocking=True)

        optimizer.zero_grad()
        logits = model(images, questions, q_lens)
        loss = criterion(logits, answers)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        total_loss += loss.item() * answers.size(0)
        correct    += (logits.argmax(1) == answers).sum().item()
        total      += answers.size(0)
    return total_loss / total, correct / total


@torch.no_grad()
def predict(model, loader, idx2answer, device, use_choices_mask: bool = True):
    """Return list[str] predicted answer strings, in dataset order.

    If `use_choices_mask` is True and a sample's question_type is multiple_choice
    AND its choices list is non-empty, the logits for all classes NOT in the
    choices are set to -inf before argmax. So the prediction is forced to be one
    of the 4 listed choices. Closed (yes/no) and open questions are unaffected.
    """
    model.eval()
    all_preds = []
    for batch in loader:
        images    = batch["image"].to(device, non_blocking=True)
        questions = batch["question"].to(device, non_blocking=True)
        q_lens    = batch["q_len"]
        choices_mask = batch["choices_mask"].to(device, non_blocking=True)
        has_choices  = batch["has_choices"]
        qtypes       = batch["question_type"]

        logits = model(images, questions, q_lens)                # [B, n_classes]

        if use_choices_mask:
            for i in range(logits.size(0)):
                if qtypes[i] == "multiple_choice" and bool(has_choices[i]):
                    logits[i] = logits[i].masked_fill(~choices_mask[i], float("-inf"))

        preds = logits.argmax(1).cpu().tolist()
        all_preds.extend(idx2answer[p] for p in preds)
    return all_preds


# === Main ============================================================
def main():
    set_seed(SEED)
    print(f"Device: {DEVICE}")

    # --- Data ---
    print("Loading data ...")
    train_ds, val_ds = get_train_val()
    test_ds = load_test()
    print(f"  train: {len(train_ds)}   val: {len(val_ds)}   test: {len(test_ds)}")

    # --- Vocabs ---
    a_vocab = get_answer_vocab()
    idx2answer = {i: a for a, i in a_vocab.items()}
    q_vocab = build_question_vocab(train_ds)
    print(f"  |answer_vocab| = {len(a_vocab)}   |question_vocab| = {len(q_vocab)}")
    with open("question_vocab.json", "w", encoding="utf-8") as f:
        json.dump(q_vocab, f, indent=2, ensure_ascii=False)

    # --- Datasets / loaders ---
    train_dataset = VQADataset(train_ds, q_vocab, a_vocab, train_transform)
    val_dataset   = VQADataset(val_ds,   q_vocab, a_vocab, eval_transform)
    test_dataset  = VQADataset(test_ds,  q_vocab, a_vocab, eval_transform)

    train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
                              num_workers=NUM_WORKERS, collate_fn=collate_fn,
                              pin_memory=(DEVICE.type == "cuda"))
    val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False,
                              num_workers=NUM_WORKERS, collate_fn=collate_fn,
                              pin_memory=(DEVICE.type == "cuda"))
```

```
    test_loader  = DataLoader(test_dataset,  batch_size=BATCH_SIZE, shuffle=False,
                              num_workers=NUM_WORKERS, collate_fn=collate_fn,
                              pin_memory=(DEVICE.type == "cuda"))

    # --- Model / optim ---
    model = CNNLSTMVQA(q_vocab_size=len(q_vocab), n_classes=len(a_vocab)).to(DEVICE)
    n_params = sum(p.numel() for p in model.parameters())
    n_train  = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"  model params: {n_params:,} total, {n_train:,} trainable")

    optimizer = torch.optim.AdamW(model.parameters(), lr=LR, weight_decay=WD)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS)
    criterion = nn.CrossEntropyLoss()

    # --- Training loop ---
    history = {"train_loss": [], "train_acc": [], "val_acc_masked": [],
               "val_acc_unmasked": [], "epoch_time_s": []}
    best_val_acc = -1.0
    print("\n--- Training ---")
    for epoch in range(1, NUM_EPOCHS + 1):
        t0 = time.time()
        tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer, criterion, DEVICE)

        # report both masked and unmasked val acc each epoch to track the gain
        val_preds_unmasked = predict(model, val_loader, idx2answer, DEVICE,
            use_choices_mask=False)
        val_preds_masked   = predict(model, val_loader, idx2answer, DEVICE,
            use_choices_mask=True)
        val_acc_unmasked = evaluate(val_preds_unmasked, val_ds)["accuracy"]
        val_acc_masked   = evaluate(val_preds_masked,   val_ds)["accuracy"]
        scheduler.step()

        dt = time.time() - t0
        history["train_loss"].append(tr_loss)
        history["train_acc"].append(tr_acc)
        history["val_acc_unmasked"].append(val_acc_unmasked)
        history["val_acc_masked"].append(val_acc_masked)
        history["epoch_time_s"].append(dt)

        print(f"Epoch {epoch:3d} | loss {tr_loss:.4f} | train_acc {tr_acc:.4f} | "
              f"val_acc {val_acc_unmasked:.4f} -> {val_acc_masked:.4f} (masked) | "
              f"{dt:.1f}s")

        if val_acc_masked > best_val_acc:
            best_val_acc = val_acc_masked
            torch.save({
                "model_state": model.state_dict(),
                "q_vocab":     q_vocab,
                "epoch":       epoch,
                "val_acc":     val_acc_masked,
            }, "cnn_lstm_best.pt")
            print(f"        ^ saved new best (val_acc {best_val_acc:.4f})")

    # --- Reload best and produce final preds ---
    ckpt = torch.load("cnn_lstm_best.pt", map_location=DEVICE)
    model.load_state_dict(ckpt["model_state"])
    print(f"\nLoaded best checkpoint from epoch {ckpt['epoch']} "
          f"(val_acc {ckpt['val_acc']:.4f})")

    print("\n" + "=" * 70)
    print("FINAL EVALUATION  (unmasked vs masked-choices)")
    print("=" * 70)

    val_unmasked  = predict(model, val_loader,  idx2answer, DEVICE, use_choices_mask=False)
    val_masked    = predict(model, val_loader,  idx2answer, DEVICE, use_choices_mask=True)
    test_unmasked = predict(model, test_loader, idx2answer, DEVICE, use_choices_mask=False)
    test_masked   = predict(model, test_loader, idx2answer, DEVICE, use_choices_mask=True)

    res_val_unmasked  = evaluate(val_unmasked,  val_ds)
    res_val_masked    = evaluate(val_masked,    val_ds)
    res_test_unmasked = evaluate(test_unmasked, test_ds)
    res_test_masked   = evaluate(test_masked,   test_ds)

    print_eval(res_val_unmasked,  title="VAL      no choice masking")
    print_eval(res_val_masked,    title="VAL      with choice masking (final)")
    print_eval(res_test_unmasked, title="TEST     no choice masking")
```

```
    print_eval(res_test_masked,   title="TEST     with choice masking (final)")

    # --- Save artifacts (the things the team report needs) ---
    with open("val_preds.json", "w", encoding="utf-8") as f:
        json.dump(val_masked, f, ensure_ascii=False)
    with open("test_preds.json", "w", encoding="utf-8") as f:
        json.dump(test_masked, f, ensure_ascii=False)
    with open("eval_val.json", "w", encoding="utf-8") as f:
        json.dump(res_val_masked, f, indent=2, ensure_ascii=False)
    with open("eval_test.json", "w", encoding="utf-8") as f:
        json.dump(res_test_masked, f, indent=2, ensure_ascii=False)
    with open("eval_val_unmasked.json", "w", encoding="utf-8") as f:
        json.dump(res_val_unmasked, f, indent=2, ensure_ascii=False)
    with open("eval_test_unmasked.json", "w", encoding="utf-8") as f:
        json.dump(res_test_unmasked, f, indent=2, ensure_ascii=False)
    with open("history.json", "w", encoding="utf-8") as f:
        json.dump(history, f, indent=2, ensure_ascii=False)

    print("\nSaved: val_preds.json, test_preds.json, eval_val.json, eval_test.json, "
          "eval_val_unmasked.json, eval_test_unmasked.json, history.json, "
          "cnn_lstm_best.pt, question_vocab.json")


if __name__ == "__main__":
    main()
```

A.2 Stacked Attention Network (`train_san.py`)

```
"""
SAN.py     Stacked Attention Network -Custom
=================================================

Architecture Implemented
------------
Image branch    : ResNet-50 ImageNet-pretrained (layer1..3 frozen,
                  layer4 fine-tuned at 10x smaller LR).
                  448x448 input -> layer4 out 2048 x 14 x 14
                  -> Linear projection -> 196 x d  (d=512)
Question branch : Word-Embedding (300) -> LSTM (hidden 1024) -> Linear -> d
Attention (x2)  : Each round projects v_I and the current query u,
                  scores 196 regions, weighted-sums, then residual-adds
                  back to u. Implements Yang et al. 2016 Eq. 11-14.
Classifier      : Dropout -> Linear(d -> num_answer_classes)

Outputs (in ./SAN_outputs/)     produced for the comparison table for models:
    val_preds.json    : list[str], predicted answers on val
    test_preds.json   : list[str], predicted answers on test
    eval_val.json     : output of common.evaluate() on val
    eval_test.json    : output of common.evaluate() on test
    history.json      : per-epoch {train_loss, val_acc, val_acc_by_qtype}
    train_loss.png
    val_accuracy.png
    train_loss_val_acc.png
    val_acc_by_qtype.png
    val_acc_closed.png
    val_acc_multiple_choice.png
    val_acc_open.png
    overall_acc_bar.png
    test_acc_by_qtype_bar.png
    test_acc_by_modality_bar.png
    san_best.pth      : best-val-acc checkpoint
    word_vocab.json   : question-side vocabulary (per-model, not shared)

To Run:
    python SAN.py
"""

import io
import json
import re
import shutil
from collections import Counter
from pathlib import Path

import matplotlib.pyplot as plt
```

```python
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from PIL import Image
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms

from common import (
    SEED, set_seed,
    get_train_val, load_test, get_answer_vocab,
    evaluate, print_eval,
)

# ============================================================================
# 0. SETUP     device, output dir, optional data-layout helper

set_seed(SEED)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE}")

PROJECT_ROOT = Path(__file__).resolve().parent
OUT_DIR      = PROJECT_ROOT / "SAN_outputs"
OUT_DIR.mkdir(exist_ok=True)
MODEL_TAG    = "SAN"


def _ensure_data_layout():
    """
    common structure between models decided by group
    """
    data_dir = PROJECT_ROOT / "data"
    folders  = ["train_1", "train_2", "train_3", "train_4", "test"]

    need_move = [f for f in folders
                 if (PROJECT_ROOT / f).is_dir()
                 and not (data_dir / f).is_dir()]
    if not need_move:
        return

    data_dir.mkdir(exist_ok=True)
    for f in need_move:
        src = PROJECT_ROOT / f
        dst = data_dir / f
        shutil.move(str(src), str(dst))
        print(f"  moved {src.name}/ -> data/{f}/")


_ensure_data_layout()


# ============================================================================
# 1. HYPERPARAMETERS

# Image / CNN
IMG_SIZE        = 448           # ResNet-50 layer4 produces 14x14 for 448 input
CNN_DIM         = 2048          # ResNet-50 layer4 channels
SPATIAL         = 14            # 14x14 = 196 spatial regions

# Question encoder
EMBED_DIM       = 300
LSTM_HIDDEN     = 1024

# Attention / projection
ATTN_DIM        = 512           # d in the SAN paper
NUM_ATTN_LAYERS = 2             # K stacked attention rounds
MAX_Q_LEN       = 20

# Training
BATCH_SIZE       = 32
NUM_EPOCHS       = 50
LR_HEAD          = 1e-4         # LSTM / attention / classifier / projection
LR_CNN           = 1e-5         # ResNet-50 layer4 fine-tune
WEIGHT_DECAY     = 1e-5
DROPOUT_P        = 0.5
PATIENCE         = 8            # early-stopping
```

```
GRAD_CLIP        = 5.0
PLATEAU_FACTOR   = 0.5
PLATEAU_PATIENCE = 3            # ReduceLROnPlateau patience


# ============================================================================
# 2. TOKENIZER + WORD VOCAB (question side     per-model, not shared with other models this part)


def tokenize(text: str):
    #Lowercase + split on non alphabetic characters.
    return re.findall(r"[a-z]+", text.lower())


def build_word_vocab(hf_dataset, min_freq: int = 1) -> dict:
    #Per-model question-side vocab built from the training questions.
    counter = Counter()
    for q in hf_dataset["question"]:
        counter.update(tokenize(q))
    vocab = {"<PAD>": 0, "<UNK>": 1}
    for w, c in counter.items():
        if c >= min_freq:
            vocab[w] = len(vocab)
    return vocab


def encode_question(question: str, vocab: dict, max_len: int = MAX_Q_LEN):
    tokens = tokenize(question)[:max_len]
    ids    = [vocab.get(t, vocab["<UNK>"]) for t in tokens]
    length = max(len(ids), 1)
    ids   += [vocab["<PAD>"]] * (max_len - len(ids))
    return ids, length


# ============================================================================
# 3. IMAGE TRANSFORMS

_NORM = transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])

TRAIN_TF = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    # NOTE: no horizontal flip     radiology images are NOT L-R symmetric Flipping teaches wrong invariances.I found experimentally
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.15, contrast=0.15),
    transforms.ToTensor(),
    _NORM,
])

EVAL_TF = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    _NORM,
])


# ============================================================================
# 4. PYTORCH DATASET

class VQADataset(Dataset):
    """
    Wraps an HF Dataset. Each __getitem__ returns:
        img    : Tensor C x H x W
        q_ids  : LongTensor MAX_Q_LEN
        q_len  : LongTensor scalar
        label  : LongTensor scalar (answer class index, or -1 if unseen-in-train)
    """

    def __init__(self, hf_ds, word_vocab: dict, answer_vocab: dict, transform):
        self.hf_ds        = hf_ds
        self.word_vocab   = word_vocab
        self.answer_vocab = answer_vocab
        self.transform    = transform

    def __len__(self):
```

```
        return len(self.hf_ds)

    def __getitem__(self, idx):
        item = self.hf_ds[idx]

        #  image
        raw = item["image"]
        if isinstance(raw, Image.Image):
            img = raw.convert("RGB")
        elif isinstance(raw, dict) and "bytes" in raw:
            img = Image.open(io.BytesIO(raw["bytes"])).convert("RGB")
        elif isinstance(raw, bytes):
            img = Image.open(io.BytesIO(raw)).convert("RGB")
        else:
            img = raw.convert("RGB")
        img = self.transform(img)

        # questions
        ids, length = encode_question(item["question"], self.word_vocab)
        q_ids = torch.tensor(ids,    dtype=torch.long)
        q_len = torch.tensor(length, dtype=torch.long)

        # labels
        label = torch.tensor(
            self.answer_vocab.get(item["answer"], -1), dtype=torch.long
        )
        return img, q_ids, q_len, label


# ============================================================================
# 5. MODEL     ResNet-50 + LSTM + Stacked Attention


class ImageEncoder(nn.Module):
    """
    ResNet-50 spatial feature extractor with partial fine-tuning.
    For 448x448 input layer4 outputs 2048 x 14 x 14.
    layer1..3 are frozen; layer4 is fine-tuned at a small LR.
    """

    def __init__(self, out_dim: int = ATTN_DIM):
        super().__init__()
        resnet = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
        self.frozen = nn.Sequential(
            resnet.conv1, resnet.bn1, resnet.relu, resnet.maxpool,
            resnet.layer1, resnet.layer2, resnet.layer3,
        )
        self.trainable = resnet.layer4
        for p in self.frozen.parameters():
            p.requires_grad = False
        self.frozen.eval()      # keep frozen BatchNorm in eval()

        self.proj = nn.Sequential(
            nn.Linear(CNN_DIM, out_dim),
            nn.Tanh(),
        )

    def train(self, mode: bool = True):
        super().train(mode)
        self.frozen.eval()      # force frozen stem to stay in eval mode
        return self

    def forward(self, x):
        with torch.no_grad():
            feat = self.frozen(x)              # B x 1024 x 28 x 28
        feat = self.trainable(feat)            # B x 2048 x 14 x 14
        B, C, H, W = feat.shape
        feat = feat.permute(0, 2, 3, 1).reshape(B, H * W, C)   # B x 196 x 2048
        return self.proj(feat)                                  # B x 196 x d


class QuestionEncoder(nn.Module):
    """Word-Embedding + 1-layer LSTM, last hidden state projected to d."""

    def __init__(self, vocab_size: int, embed_dim: int = EMBED_DIM,
                 lstm_hidden: int = LSTM_HIDDEN, out_dim: int = ATTN_DIM,
```

```
                 dropout: float = DROPOUT_P):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm  = nn.LSTM(embed_dim, lstm_hidden,
                             num_layers=1, batch_first=True)
        self.proj  = nn.Sequential(
            nn.Linear(lstm_hidden, out_dim),
            nn.Tanh(),
            nn.Dropout(dropout),
        )

    def forward(self, q_ids, q_len):
        emb    = self.embed(q_ids)                              # B x T x E
        packed = nn.utils.rnn.pack_padded_sequence(
            emb, q_len.cpu().clamp(min=1),
            batch_first=True, enforce_sorted=False,
        )
        _, (h, _) = self.lstm(packed)                           # h: 1 x B x H
        return self.proj(h.squeeze(0))                          # B x d


class AttentionLayer(nn.Module):
    #One round of soft attention over the 196 regions, residual update.

    def __init__(self, dim: int = ATTN_DIM, dropout: float = DROPOUT_P):
        super().__init__()
        self.W_I  = nn.Linear(dim, dim, bias=True)
        self.W_Q  = nn.Linear(dim, dim, bias=False)
        self.W_P  = nn.Linear(dim, 1,   bias=True)
        self.drop = nn.Dropout(dropout)

    def forward(self, v_I, u):
        h_I   = self.W_I(v_I)                                   # B x 196 x d
        h_Q   = self.W_Q(u).unsqueeze(1)                        # B x 1   x d
        h_A   = self.drop(torch.tanh(h_I + h_Q))                # B x 196 x d
        p_I   = torch.softmax(self.W_P(h_A), dim=1)             # B x 196 x 1
        u_hat = (p_I * v_I).sum(dim=1)                          # B x d
        return u_hat + u                                         # B x d residual


class SAN(nn.Module):
    #Stacked Attention Network (Yang et al., CVPR 2016).

    def __init__(self, vocab_size: int, num_classes: int,
                 attn_dim: int = ATTN_DIM, num_attn_layers: int = NUM_ATTN_LAYERS,
                 dropout: float = DROPOUT_P):
        super().__init__()
        self.img_enc    = ImageEncoder(out_dim=attn_dim)
        self.q_enc      = QuestionEncoder(vocab_size, out_dim=attn_dim, dropout=dropout)
        self.attn_stack = nn.ModuleList([
            AttentionLayer(attn_dim, dropout) for _ in range(num_attn_layers)
        ])
        self.classifier = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(attn_dim, num_classes),
        )

    def forward(self, img, q_ids, q_len):
        v_I = self.img_enc(img)                                 # B x 196 x d
        u   = self.q_enc(q_ids, q_len)                          # B x d
        for attn in self.attn_stack:
            u = attn(v_I, u)                                    # B x d
        return self.classifier(u)                               # B x num_classes


# ============================================================================
# 6. TRAINING / INFERENCE

def train_one_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss, n = 0.0, 0
    for img, q_ids, q_len, labels in loader:
        img, q_ids, q_len, labels = (
            img.to(DEVICE), q_ids.to(DEVICE),
            q_len.to(DEVICE), labels.to(DEVICE),
        )
```

```
        valid = labels >= 0
        if valid.sum().item() == 0:
            continue
        optimizer.zero_grad()
        logits = model(img, q_ids, q_len)
        loss   = criterion(logits[valid], labels[valid])
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP)
        optimizer.step()
        bs          = valid.sum().item()
        total_loss += loss.item() * bs
        n          += bs
    return total_loss / max(n, 1)


@torch.no_grad()
def predict_all(model, loader, idx2answer: dict) -> list:
    """Run the model over a loader; return list of predicted answer STRINGS."""
    model.eval()
    preds: list = []
    for img, q_ids, q_len, _ in loader:
        img   = img.to(DEVICE)
        q_ids = q_ids.to(DEVICE)
        q_len = q_len.to(DEVICE)
        logits   = model(img, q_ids, q_len)
        pred_idx = logits.argmax(1).cpu().tolist()
        preds.extend(idx2answer[p] for p in pred_idx)
    return preds


# ============================================================================
# 7. FIGURES

def _epochs(history):
    return list(range(1, len(history["train_loss"]) + 1))


def _save(fig, path):
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    plt.close(fig)
    print(f"  saved {path.name}")


def fig_train_loss(history, path):
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(_epochs(history), history["train_loss"], "b-o", markersize=4)
    ax.set_xlabel("Epoch"); ax.set_ylabel("Cross-Entropy Loss")
    ax.set_title(f"{MODEL_TAG}     Training Loss")
    ax.grid(True, alpha=0.4)
    _save(fig, path)


def fig_val_accuracy(history, path):
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(_epochs(history), [a * 100 for a in history["val_acc"]],
            "g-o", markersize=4)
    ax.set_xlabel("Epoch"); ax.set_ylabel("Validation Accuracy (%)")
    ax.set_ylim(0, 100); ax.grid(True, alpha=0.4)
    ax.set_title(f"{MODEL_TAG}     Validation Accuracy")
    _save(fig, path)


def fig_train_loss_val_acc(history, path):
    fig, ax1 = plt.subplots(figsize=(8, 4.5))
    epochs = _epochs(history)
    ax1.plot(epochs, history["train_loss"], "b-o",
             markersize=4, label="Train loss")
    ax1.set_xlabel("Epoch")
    ax1.set_ylabel("Cross-Entropy Loss", color="b")
    ax1.tick_params(axis="y", labelcolor="b")
    ax1.grid(True, alpha=0.3)

    ax2 = ax1.twinx()
    ax2.plot(epochs, [a * 100 for a in history["val_acc"]],
             "g-s", markersize=4, label="Val accuracy")
```

```
    ax2.set_ylabel("Validation Accuracy (%)", color="g")
    ax2.tick_params(axis="y", labelcolor="g")
    ax2.set_ylim(0, 100)

    ax1.set_title(f"{MODEL_TAG}     Training Loss vs Validation Accuracy")
    _save(fig, path)


def fig_val_acc_by_qtype(history, path):
    qtypes = sorted({k for d in history["val_acc_by_qtype"] for k in d})
    fig, ax = plt.subplots(figsize=(8, 4.5))
    for qt in qtypes:
        ys = [d.get(qt, 0.0) * 100 for d in history["val_acc_by_qtype"]]
        ax.plot(_epochs(history), ys, "-o", markersize=3, label=qt)
    ax.set_xlabel("Epoch"); ax.set_ylabel("Validation Accuracy (%)")
    ax.set_ylim(0, 100); ax.grid(True, alpha=0.4); ax.legend()
    ax.set_title(f"{MODEL_TAG}     Validation Accuracy per Question Type")
    _save(fig, path)


def fig_val_acc_qtype_curve(history, qtype, path, color):
    ys = [d.get(qtype, 0.0) * 100 for d in history["val_acc_by_qtype"]]
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(_epochs(history), ys, color=color, marker="o", markersize=4)
    ax.set_xlabel("Epoch"); ax.set_ylabel("Validation Accuracy (%)")
    ax.set_ylim(0, 100); ax.grid(True, alpha=0.4)
    ax.set_title(f"{MODEL_TAG}     Val accuracy on '{qtype}' questions")
    _save(fig, path)


def fig_overall_acc_bar(eval_val, eval_test, path):
    fig, ax = plt.subplots(figsize=(5, 4))
    vals = [eval_val["accuracy"] * 100, eval_test["accuracy"] * 100]
    bars = ax.bar(["Val", "Test"], vals, color=["#4C72B0", "#55A868"])
    for bar, v in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width() / 2, v + 1,
                f"{v:.1f}%", ha="center", fontsize=10)
    ax.set_ylabel("Accuracy (%)"); ax.set_ylim(0, 100)
    ax.set_title(f"{MODEL_TAG}     Overall Accuracy")
    _save(fig, path)


def fig_breakdown_bar(eval_result, key, title, path, color="#C44E52"):
    breakdown = eval_result[key]
    if not breakdown:
        return
    labels = sorted(breakdown.keys(), key=lambda k: -breakdown[k][0])
    vals   = [breakdown[k][0] * 100 for k in labels]
    ns     = [breakdown[k][1] for k in labels]

    fig, ax = plt.subplots(figsize=(max(6, len(labels) * 0.6), 4.5))
    ax.bar(range(len(labels)), vals, color=color)
    for i, (v, n) in enumerate(zip(vals, ns)):
        ax.text(i, v + 1, f"{v:.0f}%\nn={n}",
                ha="center", fontsize=8)
    ax.set_xticks(range(len(labels)))
    ax.set_xticklabels([str(l) for l in labels], rotation=30, ha="right")
    ax.set_ylabel("Accuracy (%)")
    ax.set_ylim(0, max(max(vals) * 1.20, 10))
    ax.set_title(title)
    _save(fig, path)


# ============================================================================
# 8. MAIN

def _save_json(path, obj):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(obj, f, indent=2, ensure_ascii=False)
    print(f"  saved {path.name}")


def main():
    # ---- Data --------------------------------------------------------------
    print("Loading data via common.py ...")
    train_ds, val_ds = get_train_val()
```

```
    test_ds          = load_test()
    answer_vocab     = get_answer_vocab()
    idx2answer       = {i: a for a, i in answer_vocab.items()}
    print(f"  train={len(train_ds)}   val={len(val_ds)}   test={len(test_ds)}")
    print(f"  |answer_vocab|={len(answer_vocab)}")

    # Per-model question-side vocab
    word_vocab = build_word_vocab(train_ds)
    _save_json(OUT_DIR / "word_vocab.json", word_vocab)

    # ---- PyTorch datasets / loaders ----------------------------------------
    train_set = VQADataset(train_ds, word_vocab, answer_vocab, TRAIN_TF)
    val_set   = VQADataset(val_ds,   word_vocab, answer_vocab, EVAL_TF)
    test_set  = VQADataset(test_ds,  word_vocab, answer_vocab, EVAL_TF)

    pin = DEVICE.type == "cuda"
    train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True,
                              num_workers=0, pin_memory=pin)
    val_loader   = DataLoader(val_set,   batch_size=BATCH_SIZE, shuffle=False,
                              num_workers=0, pin_memory=pin)
    test_loader  = DataLoader(test_set,  batch_size=BATCH_SIZE, shuffle=False,
                              num_workers=0, pin_memory=pin)

    # ---- Model + optimizer (2 LR groups) + plateau scheduler ---------------
    model = SAN(vocab_size=len(word_vocab),
                num_classes=len(answer_vocab)).to(DEVICE)

    cnn_ft_params = [p for p in model.img_enc.trainable.parameters()
                     if p.requires_grad]
    cnn_ft_ids    = {id(p) for p in cnn_ft_params}
    head_params   = [p for p in model.parameters()
                     if p.requires_grad and id(p) not in cnn_ft_ids]

    n_head = sum(p.numel() for p in head_params)
    n_cnn  = sum(p.numel() for p in cnn_ft_params)

    print(f"\n{'='*62}")
    print(f"  Model   : SAN     ResNet-50 backbone")
    print(f"  CNN     : layer1..3 frozen, layer4 fine-tuned")
    print(f"            layer4 -> 2048 x 14 x 14 -> Linear -> d={ATTN_DIM}")
    print(f"  Q-enc   : Embedding({len(word_vocab)}, {EMBED_DIM})  "
          f"+  LSTM(hidden={LSTM_HIDDEN})")
    print(f"  Attn    : K={NUM_ATTN_LAYERS} rounds  (d={ATTN_DIM}, "
          f"dropout={DROPOUT_P})")
    print(f"  Output  : Linear({ATTN_DIM} -> {len(answer_vocab)} classes)")
    print(f"  Params  : head={n_head:,}  cnn-ft={n_cnn:,}  "
          f"total={n_head + n_cnn:,}")
    print(f"  Opt     : Adam   lr_head={LR_HEAD}   lr_cnn={LR_CNN}   "
          f"wd={WEIGHT_DECAY}   BS={BATCH_SIZE}")
    print(f"  Sched   : ReduceLROnPlateau(max, factor={PLATEAU_FACTOR}, "
          f"patience={PLATEAU_PATIENCE})  early-stop={PATIENCE}")
    print(f"{'='*62}\n")

    optimizer = optim.Adam(
        [{"params": head_params,   "lr": LR_HEAD},
         {"params": cnn_ft_params, "lr": LR_CNN}],
        weight_decay=WEIGHT_DECAY,
    )
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="max",
        factor=PLATEAU_FACTOR, patience=PLATEAU_PATIENCE,
    )
    criterion = nn.CrossEntropyLoss(ignore_index=-1)

    # ---- Training loop -----------------------------------------------------
    history = {
        "train_loss":         [],
        "val_acc":            [],
        "val_acc_by_qtype":   [],
    }
    best_val_acc = -1.0
    patience_cnt = 0
    best_path    = OUT_DIR / "san_best.pth"

    for epoch in range(1, NUM_EPOCHS + 1):
        tr_loss   = train_one_epoch(model, train_loader, optimizer, criterion)
```

```
        val_preds = predict_all(model, val_loader, idx2answer)
        val_eval  = evaluate(val_preds, val_ds)
        vl_acc    = val_eval["accuracy"]
        scheduler.step(vl_acc)

        history["train_loss"].append(tr_loss)
        history["val_acc"].append(vl_acc)
        history["val_acc_by_qtype"].append(
            {k: v[0] for k, v in val_eval["by_question_type"].items()}
        )

        lr_head = optimizer.param_groups[0]["lr"]
        lr_cnn  = optimizer.param_groups[1]["lr"]
        qt_str = "  ".join(
            f"{k}={v[0]:.3f}" for k, v in
            sorted(val_eval["by_question_type"].items())
        )
        print(f"Epoch {epoch:3d}/{NUM_EPOCHS} | "
              f"loss={tr_loss:.4f} | val_acc={vl_acc:.4f} | "
              f"lr=[{lr_head:.1e},{lr_cnn:.1e}]")
        print(f"            {qt_str}")

        if vl_acc > best_val_acc:
            best_val_acc = vl_acc
            patience_cnt = 0
            torch.save(model.state_dict(), best_path)
            print(f"            ^ new best (saved -> {best_path.name})")
        else:
            patience_cnt += 1
            if patience_cnt >= PATIENCE:
                print(f"\nEarly stopping at epoch {epoch} "
                      f"(no improvement for {PATIENCE} epochs).")
                break

    # ---- Final inference with best checkpoint --------------------------------
    print(f"\nLoading best checkpoint: {best_path}")
    model.load_state_dict(torch.load(best_path, map_location=DEVICE))

    val_preds  = predict_all(model, val_loader,  idx2answer)
    test_preds = predict_all(model, test_loader, idx2answer)

    eval_val_result  = evaluate(val_preds,  val_ds)
    eval_test_result = evaluate(test_preds, test_ds)

    print_eval(eval_val_result,  title=f"{MODEL_TAG}     val")
    print_eval(eval_test_result, title=f"{MODEL_TAG}     test")


    print("\nSaving team-contract artifacts ...")
    _save_json(OUT_DIR / "val_preds.json",  val_preds)
    _save_json(OUT_DIR / "test_preds.json", test_preds)
    _save_json(OUT_DIR / "eval_val.json",   eval_val_result)
    _save_json(OUT_DIR / "eval_test.json",  eval_test_result)
    _save_json(OUT_DIR / "history.json",    history)

    # ---- Figures -------------------------------------------------------------
    print("\nGenerating figures ...")
    fig_train_loss(history,          OUT_DIR / "train_loss.png")
    fig_val_accuracy(history,        OUT_DIR / "val_accuracy.png")
    fig_train_loss_val_acc(history,  OUT_DIR / "train_loss_val_acc.png")
    fig_val_acc_by_qtype(history,    OUT_DIR / "val_acc_by_qtype.png")
    fig_val_acc_qtype_curve(history, "closed",
                            OUT_DIR / "val_acc_closed.png",          "#4C72B0")
    fig_val_acc_qtype_curve(history, "multiple_choice",
                            OUT_DIR / "val_acc_multiple_choice.png", "#55A868")
    fig_val_acc_qtype_curve(history, "open",
                            OUT_DIR / "val_acc_open.png",            "#C44E52")
    fig_overall_acc_bar(eval_val_result, eval_test_result,
                        OUT_DIR / "overall_acc_bar.png")
    fig_breakdown_bar(eval_test_result, "by_question_type",
                      f"{MODEL_TAG}     Test accuracy by question type",
                      OUT_DIR / "test_acc_by_qtype_bar.png",  "#8172B2")
    fig_breakdown_bar(eval_test_result, "by_modality",
                      f"{MODEL_TAG}     Test accuracy by modality",
                      OUT_DIR / "test_acc_by_modality_bar.png", "#CCB974")

```

```
    print(f"\nAll outputs are in: {OUT_DIR}")
    print("Files to hand to teammates: "
          "val_preds.json, test_preds.json, eval_val.json, "
          "eval_test.json, history.json")


if __name__ == "__main__":
    main()
```

A.3 ViT + Text Encoder model (`vit_text_vqa.py`)

```
"""
ViT + Text Encoder for medical VQA on RadImageNet-VQA.
Pretrained ViT-base (image) + DistilBERT (text) + cross-attention fusion.
Closed-set classification over the 111-class answer vocabulary.
"""

from __future__ import annotations
import argparse, json, math, time
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from PIL import Image
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms as T
from transformers import AutoTokenizer, DistilBertModel, ViTModel

from common import (SEED, evaluate, get_answer_vocab, get_train_val,
                    load_test, print_eval, set_seed)


PROJECT_ROOT = Path(__file__).resolve().parent
RUN_DIR = PROJECT_ROOT / "runs" / "vit_text"
RUN_DIR.mkdir(parents=True, exist_ok=True)

VIT_NAME = "google/vit-base-patch16-224-in21k"
TEXT_NAME = "distilbert-base-uncased"

IMAGE_SIZE = 224
MAX_TEXT_LEN = 64
FUSION_DIM = 384
NUM_HEADS = 8
NUM_FUSION_LAYERS = 2
DROPOUT = 0.1

IMNET_MEAN = (0.485, 0.456, 0.406)
IMNET_STD = (0.229, 0.224, 0.225)


class VQADataset(Dataset):
    def __init__(self, hf_ds, tokenizer, vocab, image_tf, training):
        self.ds = hf_ds
        self.tokenizer = tokenizer
        self.vocab = vocab
        self.image_tf = image_tf
        self.training = training

    def __len__(self):
        return len(self.ds)

    def __getitem__(self, idx):
        row = self.ds[idx]
        img = row["image"]
        if not isinstance(img, Image.Image):
            img = Image.fromarray(np.array(img))
        img = img.convert("RGB")
        pixel_values = self.image_tf(img)

        enc = self.tokenizer(
            row["question"], padding="max_length", truncation=True,
            max_length=MAX_TEXT_LEN, return_tensors="pt",
        )
```

```
        return {
            "pixel_values": pixel_values,
            "input_ids": enc["input_ids"].squeeze(0),
            "attention_mask": enc["attention_mask"].squeeze(0),
            "label": torch.tensor(self.vocab.get(row["answer"], -1), dtype=torch.long),
        }


def build_transforms(training):
    if training:
        return T.Compose([
            T.Resize((IMAGE_SIZE + 16, IMAGE_SIZE + 16)),
            T.RandomCrop((IMAGE_SIZE, IMAGE_SIZE)),
            T.RandomRotation(degrees=10),
            T.ColorJitter(brightness=0.1, contrast=0.1),
            T.ToTensor(),
            T.Normalize(IMNET_MEAN, IMNET_STD),
        ])
    return T.Compose([
        T.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        T.ToTensor(),
        T.Normalize(IMNET_MEAN, IMNET_STD),
    ])


class CrossAttentionBlock(nn.Module):
    # text tokens as queries, image patches as keys/values
    def __init__(self, dim, num_heads, dropout):
        super().__init__()
        self.norm_q = nn.LayerNorm(dim)
        self.norm_kv = nn.LayerNorm(dim)
        self.cross_attn = nn.MultiheadAttention(
            embed_dim=dim, num_heads=num_heads, dropout=dropout, batch_first=True)
        self.norm_ffn = nn.LayerNorm(dim)
        self.ffn = nn.Sequential(
            nn.Linear(dim, dim * 4),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(dim * 4, dim),
            nn.Dropout(dropout),
        )

    def forward(self, text, image, text_mask=None):
        q = self.norm_q(text)
        kv = self.norm_kv(image)
        attn_out, _ = self.cross_attn(q, kv, kv, need_weights=False)
        text = text + attn_out
        text = text + self.ffn(self.norm_ffn(text))
        return text


class ViTTextVQAModel(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.vit = ViTModel.from_pretrained(VIT_NAME, add_pooling_layer=False)
        self.text = DistilBertModel.from_pretrained(TEXT_NAME)

        vit_dim = self.vit.config.hidden_size
        text_dim = self.text.config.hidden_size

        self.img_proj = nn.Linear(vit_dim, FUSION_DIM)
        self.txt_proj = nn.Linear(text_dim, FUSION_DIM)

        self.fusion = nn.ModuleList([
            CrossAttentionBlock(FUSION_DIM, NUM_HEADS, DROPOUT)
            for _ in range(NUM_FUSION_LAYERS)
        ])
        self.fusion_norm = nn.LayerNorm(FUSION_DIM)

        self.classifier = nn.Sequential(
            nn.Linear(FUSION_DIM * 2, FUSION_DIM),
            nn.GELU(),
            nn.Dropout(DROPOUT),
            nn.Linear(FUSION_DIM, num_classes),
        )

```

```
    def forward(self, pixel_values, input_ids, attention_mask):
        vit_out = self.vit(pixel_values=pixel_values).last_hidden_state
        txt_out = self.text(input_ids=input_ids,
            attention_mask=attention_mask).last_hidden_state

        img = self.img_proj(vit_out)
        txt = self.txt_proj(txt_out)

        fused = txt
        for blk in self.fusion:
            fused = blk(fused, img)
        fused = self.fusion_norm(fused)

        mask = attention_mask.unsqueeze(-1).float()
        text_pool = (fused * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1.0)
        img_pool = img.mean(dim=1)

        feat = torch.cat([text_pool, img_pool], dim=-1)
        return self.classifier(feat)


@dataclass
class TrainCfg:
    epochs: int = 10
    batch_size: int = 16
    lr_backbone: float = 1e-5
    lr_head: float = 5e-4
    weight_decay: float = 0.05
    label_smoothing: float = 0.1
    warmup_ratio: float = 0.05
    num_workers: int = 2
    grad_clip: float = 1.0
    amp: bool = True


def build_param_groups(model, cfg):
    backbone_params, head_params = [], []
    for name, p in model.named_parameters():
        if not p.requires_grad:
            continue
        if name.startswith("vit.") or name.startswith("text."):
            backbone_params.append(p)
        else:
            head_params.append(p)
    return [
        {"params": backbone_params, "lr": cfg.lr_backbone, "weight_decay":
            cfg.weight_decay},
        {"params": head_params,     "lr": cfg.lr_head,     "weight_decay":
            cfg.weight_decay},
    ]


def cosine_lr(step, total, warmup, base_lrs):
    if step < warmup:
        scale = step / max(1, warmup)
    else:
        progress = (step - warmup) / max(1, total - warmup)
        scale = 0.5 * (1 + math.cos(math.pi * progress))
    return [lr * scale for lr in base_lrs]


@torch.no_grad()
def predict(model, loader, device, idx2answer, dataset=None, vocab=None):
    # If dataset+vocab given, score choice strings for MC items and emit the
    # corresponding letter. Otherwise plain argmax over 111 classes.
    model.eval()
    preds_str = []
    sample_idx = 0
    for batch in loader:
        logits = model(
            pixel_values=batch["pixel_values"].to(device, non_blocking=True),
            input_ids=batch["input_ids"].to(device, non_blocking=True),
            attention_mask=batch["attention_mask"].to(device, non_blocking=True),
        )
        bs = logits.size(0)
        argmax_idx = logits.argmax(dim=-1).cpu().tolist()
```

```
        for i in range(bs):
            pred = idx2answer.get(int(argmax_idx[i]), "<UNK>")
            if dataset is not None and vocab is not None:
                row = dataset[sample_idx + i]
                if row.get("question_type") == "multiple_choice":
                    choices = row.get("choices") or []
                    scored = [(j, vocab[c]) for j, c in enumerate(choices) if c in vocab]
                    if scored:
                        ch_logits = torch.tensor([logits[i, ci].item() for _, ci in scored])
                        best_j = scored[int(ch_logits.argmax())][0]
                        pred = chr(ord("A") + best_j)
            preds_str.append(pred)
        sample_idx += bs
    return preds_str


def train(cfg):
    set_seed(SEED)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    if device.type == "cpu":
        print("warning: CUDA not available, training on CPU")

    print(f"device: {device}")
    train_ds, val_ds = get_train_val()
    print(f"train={len(train_ds)}  val={len(val_ds)}")
    vocab = get_answer_vocab()
    idx2answer = {i: a for a, i in vocab.items()}
    num_classes = len(vocab)

    tokenizer = AutoTokenizer.from_pretrained(TEXT_NAME)
    train_set = VQADataset(train_ds, tokenizer, vocab, build_transforms(True),
        training=True)
    val_set = VQADataset(val_ds, tokenizer, vocab, build_transforms(False), training=False)

    train_loader = DataLoader(train_set, batch_size=cfg.batch_size, shuffle=True,
                              num_workers=cfg.num_workers, pin_memory=(device.type ==
                                  "cuda"))
    val_loader = DataLoader(val_set, batch_size=cfg.batch_size, shuffle=False,
                            num_workers=cfg.num_workers, pin_memory=(device.type == "cuda"))

    model = ViTTextVQAModel(num_classes=num_classes).to(device)
    n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"trainable params: {n_params/1e6:.1f}M")

    param_groups = build_param_groups(model, cfg)
    optimizer = torch.optim.AdamW(param_groups)
    base_lrs = [g["lr"] for g in param_groups]

    total_steps = cfg.epochs * len(train_loader)
    warmup_steps = int(total_steps * cfg.warmup_ratio)
    criterion = nn.CrossEntropyLoss(label_smoothing=cfg.label_smoothing)
    scaler = torch.amp.GradScaler("cuda", enabled=(cfg.amp and device.type == "cuda"))

    history = {"train_loss": [], "val_acc": [], "val_acc_by_qtype": []}
    best_acc = -1.0
    best_path = RUN_DIR / "best.pt"
    step = 0
    t0 = time.time()

    for epoch in range(1, cfg.epochs + 1):
        model.train()
        epoch_loss, epoch_n = 0.0, 0
        for batch in train_loader:
            for g, lr in zip(optimizer.param_groups,
                             cosine_lr(step, total_steps, warmup_steps, base_lrs)):
                g["lr"] = lr

            pixel_values = batch["pixel_values"].to(device, non_blocking=True)
            input_ids = batch["input_ids"].to(device, non_blocking=True)
            attention_mask = batch["attention_mask"].to(device, non_blocking=True)
            labels = batch["label"].to(device, non_blocking=True)

            optimizer.zero_grad(set_to_none=True)
            with torch.amp.autocast("cuda", enabled=(cfg.amp and device.type == "cuda")):
                logits = model(pixel_values, input_ids, attention_mask)
```

```
                loss = criterion(logits, labels)

            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
            scaler.step(optimizer)
            scaler.update()

            bs = labels.size(0)
            epoch_loss += loss.item() * bs
            epoch_n += bs
            step += 1

        train_loss = epoch_loss / max(1, epoch_n)
        # plain argmax for val (no MC fix) to keep checkpoint selection fair
        val_preds = predict(model, val_loader, device, idx2answer)
        val_result = evaluate(val_preds, val_ds)
        val_acc = val_result["accuracy"]
        history["train_loss"].append(train_loss)
        history["val_acc"].append(val_acc)
        history["val_acc_by_qtype"].append(
            {k: v[0] for k, v in val_result["by_question_type"].items()})

        elapsed = time.time() - t0
        print(f"epoch {epoch:02d}/{cfg.epochs}  loss={train_loss:.4f}  "
              f"val_acc={val_acc:.4f}  t={elapsed/60:.1f}min")

        if val_acc > best_acc:
            best_acc = val_acc
            torch.save({"model": model.state_dict(), "vocab": vocab,
                        "epoch": epoch, "val_acc": val_acc}, best_path)
            print(f"  -> saved best to {best_path}")

    with open(RUN_DIR / "history.json", "w", encoding="utf-8") as f:
        json.dump(history, f, indent=2)
    _save_plots(history)
    print(f"done. best val_acc={best_acc:.4f}  total={ (time.time()-t0)/60:.1f}min")
    return best_path


# plot styling
_FIG_SIZE = (7, 4.5)
_FIG_DPI = 140
_C_LOSS = "#1f77b4"
_C_ACC = "#ff7f0e"
_C_QT = {"closed": "#2ca02c", "multiple_choice": "#d62728", "open": "#9467bd"}
_MODEL_LABEL = "ViT + Text"


def _save_plots(history, out_dir=None):
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        return

    out_dir = out_dir or (RUN_DIR / "figures")
    out_dir.mkdir(parents=True, exist_ok=True)

    epochs = list(range(1, len(history["train_loss"]) + 1))
    train_loss = history["train_loss"]
    val_acc_pct = [100.0 * a for a in history["val_acc"]]

    # training loss
    fig, ax = plt.subplots(figsize=_FIG_SIZE)
    ax.plot(epochs, train_loss, "-o", color=_C_LOSS, markersize=5)
    ax.set_xlabel("Epoch"); ax.set_ylabel("Cross-Entropy Loss")
    ax.set_title(f"{_MODEL_LABEL}     Training Loss")
    ax.grid(alpha=0.3)
    fig.tight_layout(); fig.savefig(out_dir / "train_loss.png", dpi=_FIG_DPI);
        plt.close(fig)

    # val accuracy
    fig, ax = plt.subplots(figsize=_FIG_SIZE)
    ax.plot(epochs, val_acc_pct, "-o", color=_C_ACC, markersize=5)
```

```
    ax.set_xlabel("Epoch"); ax.set_ylabel("Accuracy (%)")
    ax.set_title(f"{_MODEL_LABEL}     Validation Accuracy")
    ax.grid(alpha=0.3)
    fig.tight_layout(); fig.savefig(out_dir / "val_accuracy.png", dpi=_FIG_DPI);
        plt.close(fig)

    # both on twin axes
    fig, ax1 = plt.subplots(figsize=_FIG_SIZE)
    ax1.plot(epochs, train_loss, "-o", color=_C_LOSS, markersize=5, label="train loss")
    ax1.set_xlabel("Epoch"); ax1.set_ylabel("Cross-Entropy Loss", color=_C_LOSS)
    ax1.tick_params(axis="y", labelcolor=_C_LOSS); ax1.grid(alpha=0.3)
    ax2 = ax1.twinx()
    ax2.plot(epochs, val_acc_pct, "-o", color=_C_ACC, markersize=5, label="val acc")
    ax2.set_ylabel("Accuracy (%)", color=_C_ACC)
    ax2.tick_params(axis="y", labelcolor=_C_ACC)
    ax1.set_title(f"{_MODEL_LABEL}     Loss & Validation Accuracy")
    fig.tight_layout(); fig.savefig(out_dir / "train_loss_val_acc.png", dpi=_FIG_DPI);
        plt.close(fig)

    qtypes_history = history.get("val_acc_by_qtype")
    if qtypes_history:
        qtypes = ["closed", "multiple_choice", "open"]
        fig, ax = plt.subplots(figsize=_FIG_SIZE)
        for qt in qtypes:
            ys = [100.0 * ep.get(qt, 0.0) for ep in qtypes_history]
            ax.plot(epochs, ys, "-o", color=_C_QT[qt], markersize=5, label=qt)
        ax.set_xlabel("Epoch"); ax.set_ylabel("Accuracy (%)")
        ax.set_title(f"{_MODEL_LABEL}     Validation Accuracy by Question Type")
        ax.grid(alpha=0.3); ax.legend()
        fig.tight_layout()
        fig.savefig(out_dir / "val_acc_by_qtype.png", dpi=_FIG_DPI); plt.close(fig)

        for qt in qtypes:
            ys = [100.0 * ep.get(qt, 0.0) for ep in qtypes_history]
            fig, ax = plt.subplots(figsize=_FIG_SIZE)
            ax.plot(epochs, ys, "-o", color=_C_QT[qt], markersize=5)
            ax.set_xlabel("Epoch"); ax.set_ylabel("Accuracy (%)")
            ax.set_title(f"{_MODEL_LABEL}     Validation Accuracy ({qt})")
            ax.grid(alpha=0.3)
            fig.tight_layout()
            fig.savefig(out_dir / f"val_acc_{qt}.png", dpi=_FIG_DPI); plt.close(fig)


def _save_eval_bars(eval_val, eval_test, out_dir=None):
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        return

    out_dir = out_dir or (RUN_DIR / "figures")
    out_dir.mkdir(parents=True, exist_ok=True)

    def _bar(labels, values, title, ylabel, color, fname):
        fig, ax = plt.subplots(figsize=_FIG_SIZE)
        bars = ax.bar(labels, values, color=color, alpha=0.85,
                      edgecolor="black", linewidth=0.6)
        ax.set_ylabel(ylabel); ax.set_title(title)
        ax.set_ylim(0, max(105, max(values) * 1.15))
        ax.grid(axis="y", alpha=0.3)
        for b, v in zip(bars, values):
            ax.text(b.get_x() + b.get_width() / 2, v + 1.5, f"{v:.1f}",
                    ha="center", fontsize=10)
        fig.tight_layout(); fig.savefig(out_dir / fname, dpi=_FIG_DPI); plt.close(fig)

    _bar(["Validation", "Test"],
         [100 * eval_val["accuracy"], 100 * eval_test["accuracy"]],
         f"{_MODEL_LABEL}     Overall Accuracy", "Accuracy (%)",
         [_C_LOSS, _C_ACC], "overall_acc_bar.png")

    qt_keys = ["closed", "multiple_choice", "open"]
    qt_vals = [100 * eval_test["by_question_type"].get(k, [0])[0] for k in qt_keys]
    _bar(qt_keys, qt_vals,
         f"{_MODEL_LABEL}     Test Accuracy by Question Type", "Accuracy (%)",
         [_C_QT[k] for k in qt_keys], "test_acc_by_qtype_bar.png")
```

```
    mod_bd = eval_test["by_modality"]
    mod_keys = sorted(mod_bd.keys())
    mod_vals = [100 * mod_bd[k][0] for k in mod_keys]
    _bar(mod_keys, mod_vals,
         f"{_MODEL_LABEL}     Test Accuracy by Modality", "Accuracy (%)",
         ["#17becf"] * len(mod_keys), "test_acc_by_modality_bar.png")


def run_predict_and_eval(checkpoint, batch_size, num_workers,
                         mc_fix=False, out_dir=None):
    set_seed(SEED)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    out_dir = Path(out_dir) if out_dir is not None else RUN_DIR
    out_dir.mkdir(parents=True, exist_ok=True)
    figs_dir = out_dir / "figures"
    tag = "mc_fix" if mc_fix else "baseline"
    print(f"device: {device}   mode: {tag}   out: {out_dir}")

    train_ds, val_ds = get_train_val()
    test_ds = load_test()

    ckpt = torch.load(checkpoint, map_location=device, weights_only=False)
    vocab = ckpt.get("vocab") or get_answer_vocab()
    idx2answer = {i: a for a, i in vocab.items()}

    tokenizer = AutoTokenizer.from_pretrained(TEXT_NAME)
    val_set = VQADataset(val_ds, tokenizer, vocab, build_transforms(False), training=False)
    test_set = VQADataset(test_ds, tokenizer, vocab, build_transforms(False),
        training=False)

    val_loader = DataLoader(val_set, batch_size=batch_size, shuffle=False,
                            num_workers=num_workers, pin_memory=(device.type == "cuda"))
    test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False,
                             num_workers=num_workers, pin_memory=(device.type == "cuda"))

    model = ViTTextVQAModel(num_classes=len(vocab)).to(device)
    model.load_state_dict(ckpt["model"])
    print(f"loaded ckpt epoch={ckpt.get('epoch')} val_acc={ckpt.get('val_acc')}")

    ds_arg = val_ds if mc_fix else None
    vc_arg = vocab if mc_fix else None
    val_preds = predict(model, val_loader, device, idx2answer, dataset=ds_arg, vocab=vc_arg)
    ds_arg = test_ds if mc_fix else None
    test_preds = predict(model, test_loader, device, idx2answer, dataset=ds_arg,
        vocab=vc_arg)

    val_result = evaluate(val_preds, val_ds)
    test_result = evaluate(test_preds, test_ds)
    print_eval(val_result, title=f"ViT+Text [{tag}] VAL")
    print_eval(test_result, title=f"ViT+Text [{tag}] TEST")

    with open(out_dir / "val_preds.json", "w", encoding="utf-8") as f:
        json.dump(val_preds, f, ensure_ascii=False)
    with open(out_dir / "test_preds.json", "w", encoding="utf-8") as f:
        json.dump(test_preds, f, ensure_ascii=False)
    with open(out_dir / "eval_val.json", "w", encoding="utf-8") as f:
        json.dump(val_result, f, indent=2, ensure_ascii=False)
    with open(out_dir / "eval_test.json", "w", encoding="utf-8") as f:
        json.dump(test_result, f, indent=2, ensure_ascii=False)
    _save_eval_bars(val_result, test_result, out_dir=figs_dir)
    print(f"wrote outputs to {out_dir}")


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--mode", choices=["train", "predict", "all"], default="all")
    p.add_argument("--epochs", type=int, default=10)
    p.add_argument("--batch_size", type=int, default=16)
    p.add_argument("--lr_backbone", type=float, default=1e-5)
    p.add_argument("--lr_head", type=float, default=5e-4)
    p.add_argument("--weight_decay", type=float, default=0.05)
    p.add_argument("--num_workers", type=int, default=2)
    p.add_argument("--no_amp", action="store_true")
    p.add_argument("--checkpoint", type=str, default=str(RUN_DIR / "best.pt"))
    p.add_argument("--mc_fix", action="store_true")
```

```
    p.add_argument("--out_dir", type=str, default=None)
    args = p.parse_args()

    cfg = TrainCfg(
        epochs=args.epochs, batch_size=args.batch_size,
        lr_backbone=args.lr_backbone, lr_head=args.lr_head,
        weight_decay=args.weight_decay, num_workers=args.num_workers,
        amp=not args.no_amp,
    )

    ckpt_path = Path(args.checkpoint)
    if args.mode in ("train", "all"):
        ckpt_path = train(cfg)

    if args.mode == "predict":
        out_dir = Path(args.out_dir) if args.out_dir else (
            RUN_DIR.parent / "vit_text_mc_fix" if args.mc_fix else RUN_DIR)
        run_predict_and_eval(ckpt_path, batch_size=args.batch_size,
                             num_workers=args.num_workers,
                             mc_fix=args.mc_fix, out_dir=out_dir)
    elif args.mode == "all":
        # produce both inference variants from the same checkpoint
        run_predict_and_eval(ckpt_path, batch_size=args.batch_size,
                             num_workers=args.num_workers,
                             mc_fix=False, out_dir=RUN_DIR)
        run_predict_and_eval(ckpt_path, batch_size=args.batch_size,
                             num_workers=args.num_workers,
                             mc_fix=True, out_dir=RUN_DIR.parent / "vit_text_mc_fix")


if __name__ == "__main__":
    main()
```

A.4 Autoencoder + VQA model (`autoencoder_vqa.py`)

```
"""
autoencoder_vqa.py       Autoencoder-VQA model for EEE443 Final Project.

Two-stage training:
  Stage 1     Train a convolutional autoencoder to reconstruct medical images
              (unsupervised, no labels required).
  Stage 2     Attach a question LSTM and MLP fusion head, then fine-tune the
              full network on the VQA classification task.

Architecture:
  Image    : ConvEncoder  ->  512-dim latent vector
  Question : Embedding + LSTM  ->  256-dim vector
  Fusion   : Concat -> Linear(768->512) -> ReLU -> Dropout -> Linear(512->111)

Usage:
  py -3.12 autoencoder_vqa.py
"""

import os
import re
import json
import time
from collections import defaultdict, Counter

# datasets must be imported before torch on Windows to avoid a PyArrow DLL conflict
from common import get_answer_vocab, get_train_val, load_test, DATA_DIR, set_seed

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

#         Constants
```

```python
NUM_CLASSES = 111
IMAGE_SIZE = 224
LATENT_DIM = 512
Q_EMBED_DIM = 128
Q_HIDDEN_DIM = 256
MAX_Q_LEN = 20

_IMAGENET_MEAN = [0.485, 0.456, 0.406]
_IMAGENET_STD = [0.229, 0.224, 0.225]


def build_question_vocab(dataset, min_freq: int = 2) -> dict:
    counter = Counter()
    for q in dataset["question"]:
        counter.update(re.findall(r"[a-z0-9]+", q.lower()))
    vocab = {"<pad>": 0, "<unk>": 1}
    for word, count in counter.most_common():
        if count >= min_freq:
            vocab[word] = len(vocab)
    return vocab


def get_image_transform(image_size: int, split: str = "val"):
    if split == "train":
        return T.Compose(
            [
                T.Lambda(lambda img: img.convert("RGB")),
                T.Resize((image_size, image_size)),
                T.RandomHorizontalFlip(),
                T.ColorJitter(brightness=0.2, contrast=0.2),
                T.ToTensor(),
                T.Normalize(_IMAGENET_MEAN, _IMAGENET_STD),
            ]
        )
    return T.Compose(
        [
            T.Lambda(lambda img: img.convert("RGB")),
            T.Resize((image_size, image_size)),
            T.ToTensor(),
            T.Normalize(_IMAGENET_MEAN, _IMAGENET_STD),
        ]
    )


class VQADataset(Dataset):
    def __init__(
        self,
        hf_dataset,
        answer_vocab: dict,
        transform,
        word2idx: dict,
        max_question_len: int = 20,
    ):
        self.data = hf_dataset
        self.answer_vocab = answer_vocab
        self.transform = transform
        self.word2idx = word2idx
        self.max_question_len = max_question_len

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]
        image = self.transform(item["image"])
        tokens = re.findall(r"[a-z0-9]+", item["question"].lower())[
            : self.max_question_len
        ]
        ids = [self.word2idx.get(t, 1) for t in tokens]
        ids += [0] * (self.max_question_len - len(ids))
        answer_idx = self.answer_vocab.get(item["answer"], 0)
        return {
            "image": image,
            "question": torch.tensor(ids, dtype=torch.long),
            "answer_idx": torch.tensor(answer_idx, dtype=torch.long),
        }
```

```python


def vqa_collate_fn(batch: list) -> dict:
    return {
        "image": torch.stack([b["image"] for b in batch]),
        "question": torch.stack([b["question"] for b in batch]),
        "answer_idx": torch.stack([b["answer_idx"] for b in batch]),
    }


def get_dataloader(dataset, batch_size: int, shuffle: bool = False) -> DataLoader:
    return DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        num_workers=0,
        collate_fn=vqa_collate_fn,
        pin_memory=torch.cuda.is_available(),
    )


#         Convolutional Encoder

class ConvEncoder(nn.Module):
    """5-layer strided conv network: 3x224x224 -> LATENT_DIM vector."""

    def __init__(self, latent_dim: int = LATENT_DIM):
        super().__init__()
        self.convs = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1),  # -> 32x112x112
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, 3, stride=2, padding=1),  # -> 64x56x56
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 128, 3, stride=2, padding=1),  # -> 128x28x28
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 256, 3, stride=2, padding=1),  # -> 256x14x14
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 512, 3, stride=2, padding=1),  # -> 512x7x7
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
        )
        self.fc = nn.Linear(512 * 7 * 7, latent_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.convs(x)
        x = x.flatten(1)
        return self.fc(x)  # (B, latent_dim)


# Convolutional Decoder
class ConvDecoder(nn.Module):
    """Mirror of ConvEncoder using transposed convolutions. Output in [0,1]."""

    def __init__(self, latent_dim: int = LATENT_DIM):
        super().__init__()
        self.fc = nn.Linear(latent_dim, 512 * 7 * 7)
        self.deconvs = nn.Sequential(
            nn.ConvTranspose2d(512, 256, 4, stride=2, padding=1),  # -> 256x14x14
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(256, 128, 4, stride=2, padding=1),  # -> 128x28x28
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(128, 64, 4, stride=2, padding=1),  # -> 64x56x56
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1),  # -> 32x112x112
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(32, 3, 4, stride=2, padding=1),  # -> 3x224x224
            nn.Sigmoid(),
        )
```

```
191
192     def forward(self, z: torch.Tensor) -> torch.Tensor:
193         x = self.fc(z)
194         x = x.view(-1, 512, 7, 7)
195         return self.deconvs(x)  # (B, 3, 224, 224)
196
197
198 # Full Autoencoder (Stage 1)
199 class ConvAutoencoder(nn.Module):
200     def __init__(self, latent_dim: int = LATENT_DIM):
201         super().__init__()
202         self.encoder = ConvEncoder(latent_dim)
203         self.decoder = ConvDecoder(latent_dim)
204
205     def forward(self, x: torch.Tensor):
206         z = self.encoder(x)
207         recon = self.decoder(z)
208         return recon, z
209
210
211 # Question Encoder
212 class QuestionEncoder(nn.Module):
213     """Embeds a token-index sequence and encodes it with an LSTM."""
214
215     def __init__(
216         self,
217         vocab_size: int,
218         embed_dim: int = Q_EMBED_DIM,
219         hidden_dim: int = Q_HIDDEN_DIM,
220     ):
221         super().__init__()
222         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
223         self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)
224
225     def forward(self, tokens: torch.Tensor) -> torch.Tensor:
226         emb = self.embedding(tokens)  # (B, L, embed_dim)
227         _, (h, _) = self.lstm(emb)  # h: (2, B, hidden_dim)
228         return torch.cat([h[0], h[1]], dim=1)  # (B, 2*hidden_dim)
229
230
231 # Autoencoder-VQA model (Stage 2)
232 class AutoencoderVQA(nn.Module):
233     """
234     Full VQA model built on top of the pretrained ConvEncoder.
235     Image encoder weights loaded from Stage 1, fine-tuned jointly
236     with the question encoder and fusion head.
237     """
238
239     def __init__(
240         self,
241         encoder: ConvEncoder,
242         vocab_size: int,
243         latent_dim: int = LATENT_DIM,
244         q_embed: int = Q_EMBED_DIM,
245         q_hidden: int = Q_HIDDEN_DIM,
246         num_classes: int = NUM_CLASSES,
247     ):
248         super().__init__()
249         self.image_encoder = encoder
250         self.question_encoder = QuestionEncoder(vocab_size, q_embed, q_hidden)
251         self.fusion = nn.Sequential(
252             nn.Linear(latent_dim + q_hidden * 2, 512),
253             nn.ReLU(inplace=True),
254             nn.Dropout(p=0.3),
255             nn.Linear(512, num_classes),
256         )
257
258     def forward(
259         self, images: torch.Tensor, question_tokens: torch.Tensor
260     ) -> torch.Tensor:
261         img_feat = self.image_encoder(images)  # (B, latent_dim)
262         q_feat = self.question_encoder(question_tokens)  # (B, q_hidden)
263         fused = torch.cat([img_feat, q_feat], dim=1)
264         return self.fusion(fused)  # (B, num_classes)
265
266
```

```python
# Image-only dataset for Stage 1
class ImageOnlyDataset(Dataset):
    """No labels      only images. Values in [0,1] to match Sigmoid decoder."""

    _transform = T.Compose(
        [
            T.Lambda(lambda img: img.convert("RGB")),
            T.Resize((IMAGE_SIZE, IMAGE_SIZE)),
            T.ToTensor(),
        ]
    )

    def __init__(self, hf_dataset):
        self.data = hf_dataset

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self._transform(self.data[idx]["image"])


# Dataset with question_type field (for detailed evaluation)
class VQADatasetWithQType(VQADataset):
    def __getitem__(self, idx):
        item = super().__getitem__(idx)
        qtype = self.data[idx].get("question_type", "unknown") or "unknown"
        meta = self.data[idx].get("metadata") or {}
        modality = (meta.get("modality") or "unknown").strip() or "unknown"
        item["question_type"] = qtype
        item["modality"] = modality
        return item


def collate_with_qtype(batch: list) -> dict:
    base = vqa_collate_fn(batch)
    base["question_type"] = [b["question_type"] for b in batch]
    base["modality"] = [b["modality"] for b in batch]
    return base


# Stage 1: train autoencoder
def train_autoencoder(
    model: ConvAutoencoder,
    hf_dataset,
    num_epochs: int = 30,
    lr: float = 1e-3,
    batch_size: int = 32,
    device: str = "cpu",
    save_path: str = "autoencoder.pth",
) -> list:
    """Trains with MSE reconstruction loss. Returns per-epoch loss list."""
    model.to(device)
    dataset = ImageOnlyDataset(hf_dataset)
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
        num_workers=0,
        pin_memory=(device != "cpu"),
    )
    optimizer = optim.Adam(model.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.5)
    criterion = nn.MSELoss()
    losses = []

    print(f"\n{'    '*55}")
    print(f"  Stage 1     Autoencoder Pretraining  ({len(dataset)} images)")
    print(f"{'    '*55}")

    for epoch in range(1, num_epochs + 1):
        model.train()
        epoch_loss = 0.0
        for images in loader:
            images = images.to(device)
            recon, _ = model(images)
```

```
            loss = criterion(recon, images)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()

        scheduler.step()
        avg = epoch_loss / len(loader)
        losses.append(avg)
        print(f"  Epoch {epoch:3d}/{num_epochs}  Recon Loss: {avg:.5f}")

    torch.save(model.state_dict(), save_path)
    print(f"\n  Autoencoder weights saved -> {save_path}")
    return losses


# Stage 2: fine-tune VQA
def train_vqa(
    model: AutoencoderVQA,
    train_loader: DataLoader,
    val_loader: DataLoader,
    num_epochs: int = 30,
    lr: float = 1e-4,
    device: str = "cpu",
    save_path: str = "autoencoder_vqa.pth",
) -> tuple:
    """Fine-tunes with cross-entropy loss. Returns (train_losses, val_accs)."""
    model.to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs)
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

    train_losses, val_accs = [], []
    best_val_acc = 0.0

    print(f"\n{'   '*55}")
    print(f"  Stage 2     VQA Fine-tuning")
    print(f"{'   '*55}")

    for epoch in range(1, num_epochs + 1):
        model.train()
        epoch_loss = 0.0
        for batch in train_loader:
            images = batch["image"].to(device)
            questions = batch["question"].to(device)
            answers = batch["answer_idx"].to(device)

            logits = model(images, questions)
            loss = criterion(logits, answers)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()

        scheduler.step()
        avg_loss = epoch_loss / len(train_loader)
        val_acc = evaluate(model, val_loader, device)

        train_losses.append(avg_loss)
        val_accs.append(val_acc)

        flag = ""
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), save_path)
            flag = "  <- best"

        print(
            f"  Epoch {epoch:3d}/{num_epochs}  "
            f"Loss: {avg_loss:.4f}  Val Acc: {val_acc*100:.2f}%{flag}"
        )

    print(f"\n  Best Val Acc: {best_val_acc*100:.2f}%")
    print(f"  Best model saved -> {save_path}")

    hist_path = os.path.join(DATA_DIR, "history.json")
```

```
    with open(hist_path, "w") as f:
        json.dump({"train_loss": train_losses, "val_acc": val_accs}, f, indent=2)
    print(f"  Training history saved -> {hist_path}")
    return train_losses, val_accs


# Simple evaluation (used during training)
@torch.no_grad()
def evaluate(model: AutoencoderVQA, loader: DataLoader, device: str = "cpu") -> float:
    model.eval()
    correct = total = 0
    for batch in loader:
        images = batch["image"].to(device)
        questions = batch["question"].to(device)
        answers = batch["answer_idx"].to(device)
        preds = model(images, questions).argmax(dim=1)
        correct += (preds == answers).sum().item()
        total += answers.size(0)
    return correct / total if total else 0.0


# Detailed evaluation (used after training)
def _answer_category(answer: str) -> str:
    if answer in ("yes", "no"):
        return "yes/no"
    if answer in ("A", "B", "C", "D"):
        return "multiple choice"
    return "open-ended (medical)"


@torch.no_grad()
def evaluate_detailed(
    model: AutoencoderVQA, loader: DataLoader, device: str, idx2answer: dict
) -> tuple:
    """
    Returns (overall_acc, by_modality, by_category, all_preds, all_targets).
    Loader must include 'question_type' and 'modality' keys in each batch.
    """
    model.eval()
    overall_correct = overall_total = 0
    by_modality = defaultdict(lambda: {"correct": 0, "total": 0})
    by_category = defaultdict(lambda: {"correct": 0, "total": 0})
    all_preds, all_targets = [], []

    for batch in loader:
        images = batch["image"].to(device)
        questions = batch["question"].to(device)
        answers = batch["answer_idx"].to(device)
        modalities = batch["modality"]

        preds = model(images, questions).argmax(dim=1)

        for i in range(len(answers)):
            pred_idx = preds[i].item()
            target_idx = answers[i].item()
            correct = pred_idx == target_idx

            target_str = idx2answer.get(target_idx, "unknown")
            modality = modalities[i] if modalities[i] else "unknown"
            category = _answer_category(target_str)

            overall_correct += int(correct)
            overall_total += 1

            by_modality[modality]["correct"] += int(correct)
            by_modality[modality]["total"] += 1
            by_category[category]["correct"] += int(correct)
            by_category[category]["total"] += 1

            all_preds.append(idx2answer.get(pred_idx, "unknown"))
            all_targets.append(target_str)

    overall_acc = overall_correct / overall_total if overall_total else 0.0
    return overall_acc, dict(by_modality), dict(by_category), all_preds, all_targets


```

```python
# Plotting
def plot_ae_loss(losses: list, save_path: str = "ae_loss.png"):
    plt.figure(figsize=(8, 4))
    plt.plot(range(1, len(losses) + 1), losses, marker="o", linewidth=1.5)
    plt.xlabel("Epoch")
    plt.ylabel("MSE Reconstruction Loss")
    plt.title("Stage 1     Autoencoder Training Loss")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(save_path, dpi=150)
    plt.close()
    print(f"  Autoencoder loss plot saved -> {save_path}")


def plot_vqa_curves(
    train_losses: list, val_accs: list, save_path: str = "vqa_curves.png"
):
    epochs = range(1, len(train_losses) + 1)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(epochs, train_losses, marker="o", linewidth=1.5, color="steelblue")
    ax1.set_xlabel("Epoch")
    ax1.set_ylabel("Cross-Entropy Loss")
    ax1.set_title("Stage 2     Training Loss")
    ax1.grid(True)

    ax2.plot(
        epochs,
        [a * 100 for a in val_accs],
        marker="o",
        linewidth=1.5,
        color="darkorange",
    )
    ax2.set_xlabel("Epoch")
    ax2.set_ylabel("Accuracy (%)")
    ax2.set_title("Stage 2     Validation Accuracy")
    ax2.grid(True)

    plt.tight_layout()
    plt.savefig(save_path, dpi=150)
    plt.close()
    print(f"  VQA training curves saved -> {save_path}")


def plot_reconstructions(
    model: ConvAutoencoder,
    hf_dataset,
    n: int = 6,
    device: str = "cpu",
    save_path: str = "reconstructions.png",
):
    """Side-by-side original vs reconstructed grid for the report."""
    model.eval()
    ds = ImageOnlyDataset(hf_dataset)
    indices = np.random.choice(len(ds), n, replace=False)
    originals = torch.stack([ds[i] for i in indices]).to(device)

    with torch.no_grad():
        recons, _ = model(originals)

    originals = originals.cpu()
    recons = recons.cpu()

    fig, axes = plt.subplots(2, n, figsize=(n * 2.5, 5))
    for col in range(n):
        for row, imgs in enumerate([originals, recons]):
            img = imgs[col].permute(1, 2, 0).numpy().clip(0, 1)
            axes[row, col].imshow(img)
            axes[row, col].axis("off")
    axes[0, 0].set_title("Original", fontsize=9, loc="left")
    axes[1, 0].set_title("Reconstructed", fontsize=9, loc="left")
    plt.suptitle("Autoencoder Reconstructions", fontsize=11)
    plt.tight_layout()
    plt.savefig(save_path, dpi=150)
    plt.close()
    print(f"  Reconstruction grid saved -> {save_path}")
```

```python


def plot_evaluation_bars(
    by_category: dict, by_modality: dict, save_path: str = "evaluation_results.png"
):
    labels_cat = list(by_category.keys())
    accs_cat = [100 * v["correct"] / v["total"] for v in by_category.values()]
    labels_mod = list(by_modality.keys())
    accs_mod = [100 * v["correct"] / v["total"] for v in by_modality.values()]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

    bars1 = ax1.bar(labels_cat, accs_cat, color=["steelblue", "darkorange", "seagreen"])
    ax1.set_ylabel("Accuracy (%)")
    ax1.set_title("Accuracy by Answer Category")
    ax1.set_ylim(0, 100)
    ax1.grid(axis="y", linestyle="--", alpha=0.6)
    for bar, val in zip(bars1, accs_cat):
        ax1.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 1,
            f"{val:.1f}%",
            ha="center",
            fontsize=9,
        )

    colors2 = plt.cm.tab10(np.linspace(0, 0.7, len(labels_mod)))
    bars2 = ax2.bar(labels_mod, accs_mod, color=colors2)
    ax2.set_ylabel("Accuracy (%)")
    ax2.set_title("Accuracy by Imaging Modality")
    ax2.set_ylim(0, 100)
    ax2.tick_params(axis="x", rotation=20)
    ax2.grid(axis="y", linestyle="--", alpha=0.6)
    for bar, val in zip(bars2, accs_mod):
        ax2.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 1,
            f"{val:.1f}%",
            ha="center",
            fontsize=9,
        )

    plt.suptitle("Autoencoder-VQA       Test Set Evaluation", fontsize=12)
    plt.tight_layout()
    plt.savefig(save_path, dpi=150)
    plt.close()
    print(f"  Evaluation bar chart saved -> {save_path}")


# Main
def main():
    set_seed(42)
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Device: {device}")

    # Data
    print("\nLoading datasets...")
    train_split, val_split = get_train_val()
    test_hf = load_test()
    print(f"  Train / Val : {len(train_split)} / {len(val_split)}")
    print(f"  Test samples: {len(test_hf)}")

    # Vocabulary
    print("\nBuilding question vocabulary...")
    word2idx = build_question_vocab(train_split, min_freq=2)
    answer_vocab = get_answer_vocab()
    idx2answer = {v: k for k, v in answer_vocab.items()}
    print(f"  Vocab size: {len(word2idx)}")

    t_total_start = time.time()

    # Stage 1: Autoencoder
    autoencoder = ConvAutoencoder(latent_dim=LATENT_DIM)
    ae_path = os.path.join(DATA_DIR, "autoencoder.pth")

    t1 = time.time()
```

```
    if os.path.exists(ae_path):
        print(f"\nFound existing autoencoder weights, skipping Stage 1.")
        autoencoder.load_state_dict(torch.load(ae_path, map_location=device))
        t_stage1 = 0.0
    else:
        ae_losses = train_autoencoder(
            autoencoder,
            train_split,
            num_epochs=30,
            lr=1e-3,
            batch_size=32,
            device=device,
            save_path=ae_path,
        )
        t_stage1 = time.time() - t1
        plot_ae_loss(ae_losses, os.path.join(DATA_DIR, "ae_loss.png"))
        print(f"  Stage 1 time: {t_stage1/60:.1f} min")

    plot_reconstructions(
        autoencoder.to(device),
        val_split,
        n=6,
        device=device,
        save_path=os.path.join(DATA_DIR, "reconstructions.png"),
    )

    # Stage 2: VQA fine-tuning
    train_tf = get_image_transform(IMAGE_SIZE, split="train")
    val_tf = get_image_transform(IMAGE_SIZE, split="val")

    train_ds = VQADataset(
        train_split,
        answer_vocab,
        train_tf,
        word2idx=word2idx,
        max_question_len=MAX_Q_LEN,
    )
    val_ds = VQADataset(
        val_split, answer_vocab, val_tf, word2idx=word2idx, max_question_len=MAX_Q_LEN
    )

    train_loader = get_dataloader(train_ds, batch_size=32, shuffle=True)
    val_loader = get_dataloader(val_ds, batch_size=64, shuffle=False)

    vqa_model = AutoencoderVQA(
        encoder=autoencoder.encoder,
        vocab_size=len(word2idx),
        latent_dim=LATENT_DIM,
        num_classes=NUM_CLASSES,
    )

    vqa_path = os.path.join(DATA_DIR, "autoencoder_vqa.pth")
    t2 = time.time()
    if os.path.exists(vqa_path):
        print(f"\nFound existing VQA model weights, skipping Stage 2.")
        vqa_model.load_state_dict(torch.load(vqa_path, map_location=device))
        train_losses, val_accs = [], []
        t_stage2 = 0.0
    else:
        train_losses, val_accs = train_vqa(
            vqa_model,
            train_loader,
            val_loader,
            num_epochs=30,
            lr=1e-4,
            device=device,
            save_path=vqa_path,
        )
        t_stage2 = time.time() - t2
        print(f"  Stage 2 time: {t_stage2/60:.1f} min")
        plot_vqa_curves(
            train_losses, val_accs, os.path.join(DATA_DIR, "vqa_curves.png")
        )

    # Detailed test evaluation
    print("\nRunning detailed evaluation on test set...")
```

```
    vqa_model.load_state_dict(torch.load(vqa_path, map_location=device))

    test_ds = VQADatasetWithQType(
        test_hf, answer_vocab, val_tf, word2idx=word2idx, max_question_len=MAX_Q_LEN
    )
    test_loader = DataLoader(
        test_ds,
        batch_size=64,
        shuffle=False,
        num_workers=0,
        collate_fn=collate_with_qtype,
        pin_memory=(device == "cuda"),
    )

    # val evaluation for JSON export
    print("Running detailed evaluation on validation set...")
    val_ds_qt = VQADatasetWithQType(
        val_split, answer_vocab, val_tf, word2idx=word2idx, max_question_len=MAX_Q_LEN
    )
    val_loader_qt = DataLoader(
        val_ds_qt,
        batch_size=64,
        shuffle=False,
        num_workers=0,
        collate_fn=collate_with_qtype,
        pin_memory=(device == "cuda"),
    )
    val_acc, val_by_modality, val_by_category, val_preds, val_targets = (
        evaluate_detailed(vqa_model, val_loader_qt, device, idx2answer)
    )

    t3 = time.time()
    overall_acc, by_modality, by_category, test_preds, test_targets = evaluate_detailed(
        vqa_model, test_loader, device, idx2answer
    )
    t_eval = time.time() - t3
    t_total = time.time() - t_total_start

    # Save JSON files
    def make_eval_dict(acc, by_cat, by_mod):
        return {
            "overall_acc": round(acc * 100, 4),
            "by_category": {
                k: {
                    "correct": v["correct"],
                    "total": v["total"],
                    "acc": round(100 * v["correct"] / v["total"], 4),
                }
                for k, v in by_cat.items()
            },
            "by_modality": {
                k: {
                    "correct": v["correct"],
                    "total": v["total"],
                    "acc": round(100 * v["correct"] / v["total"], 4),
                }
                for k, v in by_mod.items()
            },
        }

    with open(os.path.join(DATA_DIR, "eval_val.json"), "w") as f:
        json.dump(
            make_eval_dict(val_acc, val_by_category, val_by_modality), f, indent=2
        )
    with open(os.path.join(DATA_DIR, "eval_test.json"), "w") as f:
        json.dump(make_eval_dict(overall_acc, by_category, by_modality), f, indent=2)
    with open(os.path.join(DATA_DIR, "val_preds.json"), "w") as f:
        json.dump(
            [{"pred": p, "target": t} for p, t in zip(val_preds, val_targets)],
            f,
            indent=2,
        )
    with open(os.path.join(DATA_DIR, "test_preds.json"), "w") as f:
        json.dump(
            [{"pred": p, "target": t} for p, t in zip(test_preds, test_targets)],
            f,
```

```
            indent=2,
        )
    print(
        "  JSON files saved -> eval_val.json, eval_test.json, val_preds.json,
            test_preds.json"
    )

    # Print & save results
    def acc_str(d):
        c, t = d["correct"], d["total"]
        return f"{c}/{t}  ({100*c/t:.2f}%)" if t else "0/0"

    sep = "=" * 55
    lines = [
        sep,
        "  Autoencoder-VQA  --  Test Set Results",
        sep,
        f"\nOverall Accuracy : {overall_acc*100:.2f}%",
        "\n-- By Answer Category --",
    ]
    for cat, d in sorted(by_category.items()):
        lines.append(f"  {cat:<28} {acc_str(d)}")
    lines.append("\n-- By Imaging Modality --")
    for mod, d in sorted(by_modality.items()):
        lines.append(f"  {mod:<28} {acc_str(d)}")
    lines.append("\n-- Runtime --")
    if t_stage1 > 0:
        lines.append(f"  Stage 1 (autoencoder)  : {t_stage1/60:.1f} min")
    lines.append(f"  Stage 2 (VQA training) : {t_stage2/60:.1f} min")
    lines.append(f"  Evaluation             : {t_eval:.1f} sec")
    lines.append(f"  Total                  : {t_total/60:.1f} min")
    lines.append(sep)

    report = "\n".join(lines)
    print(report)

    txt_path = os.path.join(DATA_DIR, "evaluation_results.txt")
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write(report + "\n")
    print(f"\n  Text report saved -> {txt_path}")

    plot_evaluation_bars(
        by_category, by_modality, os.path.join(DATA_DIR, "evaluation_results.png")
    )


if __name__ == "__main__":
    main()
```

A.5 Shared utilities (common.py)

```
"""
EEE 443/543 NEURAL NETWORKS     FINAL PROJECT
Shared utilities for our 4-person group. Single file, drop it next to the
zips and import from it. This docstring also serves as the project context
for AI coding assistants     read it before suggesting code.

================================================================================
WHAT WE ARE BUILDING
================================================================================
Visual Question Answering (VQA) on the RadImageNet-VQA dataset:
medical / radiology images paired with multiple-choice questions.
Each example: image + question + list of choices + one correct answer
string + question_type + metadata (modality, pathology, location, etc.).

================================================================================
TEAM AND MODELS  (4 members, one model each)
================================================================================
  Member 1     CNN-LSTM             : CNN image encoder + LSTM text encoder.
                                      Classic VQA baseline.
  Member 2     ViT + Text Encoder   : Vision Transformer + Transformer text
                                      encoder. SOTA-style.
  Member 3     Autoencoder pretrain : Image autoencoder pretraining, then
                                      fine-tune on VQA. Enhancement.
  Member 4     Conditional GAN aug  : cGAN to oversample rare classes;
```

```
                                addresses class imbalance.

All 4 models train on the same combined training set and are evaluated
with the same metric function so the final-report comparison is fair.

============================================================================
DATASET LAYOUT  (zips already in the project root)
============================================================================
  train_1.zip ... train_4.zip   : 4 shards, ~1800 each -> ~7200 train total
  test.zip                      : ~1800 held-out examples
  (Original RadImageNet-VQA benchmark = 9000; we use 80/20 train/test.)
  After our 90/10 train-val split inside the 7200 train set:
      train ~6480   val ~720   test ~1800

NOTE: not all examples have populated `choices`     for `question_type ==
"closed"` (yes/no) and `"open"` (free-form) it is often None/[]. Only
`question_type == "multiple_choice"` is guaranteed to have choices.

Each zip contains a HuggingFace `Dataset` saved with `save_to_disk`
(Apache Arrow format). Per-example schema:

  image          : PIL image
  question       : str
  choices        : list[str]
  answer         : str   (one of `choices`     the correct one)
  question_type  : str
  metadata       : { modality, pathology, location, is_abnormal,
                     content_type, correct_text, question_id }

============================================================================
KEY FINDINGS FROM RUNNING common.py ON THE DATA  (read these!)
============================================================================
1. The training set is SMALL (~7200, after split: 6480 train / 720 val).
   Overfitting risk is high. This makes the Conditional GAN augmentation
   model and the Autoencoder pretraining model especially valuable
   they are not just "extras", they directly address the data-scarcity
   problem the CNN-LSTM and ViT+Text Encoder models will hit.

2. Three `question_type` values in the data, with very different
   difficulty profiles (val-set distribution + majority-class baseline):
       closed           416/720   baseline acc 55.8%   (yes/no)
       multiple_choice  158/720   baseline acc  0.0%
       open             146/720   baseline acc  0.0%
   Always REPORT PER-QUESTION-TYPE ACCURACY (evaluate() already does
   this)     a single overall number hides where each model wins/loses.

3. Answer vocabulary is small: only ~111 unique answer strings across
   the entire training set. Closed-set classification over this vocab
   is a viable default for ALL FOUR models (CNN-LSTM, ViT+Text Encoder,
   Autoencoder pretrain, Conditional GAN aug). Even "open" questions
   collapse into this small vocab.

4. The `choices` field is NOT always populated. It is None / [] for
   most `closed` and `open` examples, and is only guaranteed to be
   populated for `question_type == "multiple_choice"`. If you implement
   multiple-choice scoring (scoring each choice and picking argmax),
   you must fall back to closed-set classification when choices is
   missing.

============================================================================
FRAMEWORK
============================================================================
PyTorch. Required: torch, torchvision, datasets, pillow, numpy.
Members 2 and 3 will likely also use: transformers, timm.

============================================================================
QUICK USAGE
============================================================================
    from common import (set_seed, get_train_val, load_test,
                        get_answer_vocab, evaluate, print_eval)

    set_seed()
    train_ds, val_ds = get_train_val()
    test_ds = load_test()
    vocab = get_answer_vocab()              # answer_string -> class_idx
    idx2answer = {i: a for a, i in vocab.items()}
```

```
    # ... train your model, produce predicted answer STRINGS ...
    result = evaluate(val_preds, val_ds)
    print_eval(result, title="My Model     val")

Run `python common.py` once to extract the zips, build the answer vocab,
and sanity-check with a majority-class baseline. Then SHARE the generated
`answer_vocab.json` with the team so every member gets identical class
indices (do not let each member rebuild it).

============================================================================
DO NOT CHANGE  (would break cross-model comparability)
============================================================================
  - SEED                (= 42)
  - VAL_RATIO           (= 0.10)
  - the train/val split logic
  - the shared answer_vocab.json
  - evaluate() signature and metrics

WHAT EACH MEMBER OWNS  (model-specific, free choice):
  - Image transforms (resize / normalize / augmentation)
  - Text tokenization (custom vocab vs. pretrained tokenizer)
  - Architecture, optimizer, LR schedule, training loop, checkpoints
  - Per-model logging, loss/accuracy curves, ablations

============================================================================
WHAT EVERY MEMBER PRODUCES  (for the report comparison table)
============================================================================
  val_preds.json    list[str], predicted answers, len == len(val_ds)
  test_preds.json   list[str], predicted answers, len == len(test_ds)
  eval_val.json     output of common.evaluate() on val
  eval_test.json    output of common.evaluate() on test

============================================================================
MODELING NOTES
============================================================================
  - Default: closed-set classification over `answer_vocab`. Predict a
    class index, map back to string via idx2answer for evaluate().
  - Alternative: multiple-choice scoring (score each of `choices`,
    pick argmax). Better for unseen answers. Still produce STRINGS.
  - Member 4 (cGAN): inspect distribution of `answer`,
    `metadata.pathology`, `metadata.modality` on train -> imbalance axes.

============================================================================
DEADLINE
============================================================================
  Final report due 2026-05-16, 17:00. 10 pages + appendix (with code).
  Sections: Abstract / Introduction / Methods / Results / Discussion /
  References. PyTorch or TensorFlow allowed; bare MLP/CNN disallowed.
"""

import json
import random
import zipfile
from collections import Counter
from pathlib import Path

import numpy as np
from datasets import concatenate_datasets, load_from_disk

try:
    import torch
    _HAS_TORCH = True
except ImportError:
    _HAS_TORCH = False


# === Constants     DO NOT CHANGE without team consensus ======================
PROJECT_ROOT = Path(__file__).resolve().parent
DATA_DIR     = PROJECT_ROOT / "data"
VOCAB_PATH   = PROJECT_ROOT / "answer_vocab.json"

SEED       = 42
VAL_RATIO  = 0.10

TRAIN_ZIPS = ["train_1.zip", "train_2.zip", "train_3.zip", "train_4.zip"]
```

```python
TEST_ZIP   = "test.zip"


# === Reproducibility ======================================================
def set_seed(seed: int = SEED):
    random.seed(seed)
    np.random.seed(seed)
    if _HAS_TORCH:
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)


# === Data loading =========================================================
def _extract_zip(zip_name: str) -> Path:
    """Extract <zip_name> into ./data/ (idempotent). Returns the dataset dir."""
    inner_name = zip_name.replace(".zip", "")
    target = DATA_DIR / inner_name
    if target.exists() and any(target.iterdir()):
        return target
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    with zipfile.ZipFile(PROJECT_ROOT / zip_name) as z:
        z.extractall(DATA_DIR)
    return target


def load_train_full():
    """Concatenate all 4 training shards into one Dataset (~36k examples)."""
    parts = [load_from_disk(str(_extract_zip(z))) for z in TRAIN_ZIPS]
    return concatenate_datasets(parts)


def load_test():
    """Load the held-out test set (~9k examples)."""
    return load_from_disk(str(_extract_zip(TEST_ZIP)))


def get_train_val(val_ratio: float = VAL_RATIO, seed: int = SEED):
    """
    Fixed train/val split. Same split for all team members.
    Returns (train_ds, val_ds).
    """
    full = load_train_full()
    split = full.train_test_split(test_size=val_ratio, seed=seed)
    return split["train"], split["test"]


# === Answer label space (closed-set classification) =======================
#
# Why answer_vocab.json exists and why it must be SHARED, not regenerated:
# ------------------------------------------------------------------------
# The dataset's `answer` field is a string ("yes", "no", "pneumonia", ...).
# To train a classifier, every model needs to map that string to an integer
# class index. If each team member rebuilds this mapping locally, the
# Counter iteration order, dict insertion order, or filtering threshold can
# silently produce DIFFERENT integer indices for the same answer string.
# That makes class-balanced losses, confusion matrices, and any saved
# prediction tensors impossible to compare across our 4 models.
#
# So: ONE person runs `python common.py` once, the file `answer_vocab.json`
# is generated (~111 classes for RadImageNet-VQA), and that exact file is
# shared with the other 3 members alongside common.py. After that
# get_answer_vocab() just loads it from disk on every machine.
def build_answer_vocab(min_count: int = 1, save: bool = True) -> dict:
    """
    Build answer-string -> class-index mapping from the FULL training set.
    Cached to answer_vocab.json     once built, every member loads the same map.
    Send answer_vocab.json to all team members so nobody rebuilds it.
    """
    if VOCAB_PATH.exists():
        with open(VOCAB_PATH, "r", encoding="utf-8") as f:
            return json.load(f)

    full = load_train_full()
    counts = Counter(full["answer"])
    answers = sorted(a for a, c in counts.items() if c >= min_count)
    vocab = {a: i for i, a in enumerate(answers)}
```

```
    if save:
        with open(VOCAB_PATH, "w", encoding="utf-8") as f:
            json.dump(vocab, f, indent=2, ensure_ascii=False)
    return vocab


def get_answer_vocab() -> dict:
    """Returns the cached answer->idx mapping (builds it once if missing)."""
    return build_answer_vocab()


# === Evaluation     canonical metric. Use this for the final report. ========
def evaluate(predictions, dataset, idx2answer: dict = None) -> dict:
    """
    Compute the agreed-upon metrics so the 4 models are directly comparable.

    Args:
        predictions : list of predicted answer STRINGS, length == len(dataset).
                      If your model outputs class indices instead, pass
                      idx2answer = {idx: answer_string} and predictions as ints.
        dataset     : HF Dataset with columns answer, question_type, metadata.
        idx2answer  : optional int->str map; only needed if preds are ints.

    Returns:
        dict { accuracy, n, by_question_type, by_modality }
        where the breakdown values are (acc, n) tuples.
    """
    assert len(predictions) == len(dataset), (
        f"len(predictions)={len(predictions)} but len(dataset)={len(dataset)}"
    )

    if idx2answer is not None:
        predictions = [idx2answer.get(int(p), "<UNK>") for p in predictions]

    gold     = dataset["answer"]
    qtype    = dataset["question_type"]
    modality = [m["modality"] for m in dataset["metadata"]]

    correct = [int(p == g) for p, g in zip(predictions, gold)]
    overall = sum(correct) / len(correct)

    def _grouped(keys):
        groups: dict = {}
        for k, c in zip(keys, correct):
            groups.setdefault(k, []).append(c)
        return {k: (sum(v) / len(v), len(v)) for k, v in groups.items()}

    return {
        "accuracy":         overall,
        "n":                len(correct),
        "by_question_type": _grouped(qtype),
        "by_modality":      _grouped(modality),
    }


def print_eval(result: dict, title: str = ""):
    print(f"\n=== {title or 'Evaluation'} ===")
    print(f"Overall accuracy: {result['accuracy']:.4f}   (n={result['n']})")
    print("\nBy question_type:")
    for k, (acc, n) in sorted(result["by_question_type"].items()):
        print(f"  {str(k):<32s} acc={acc:.4f}  n={n}")
    print("\nBy modality:")
    for k, (acc, n) in sorted(result["by_modality"].items()):
        print(f"  {str(k):<32s} acc={acc:.4f}  n={n}")


# === Sanity check: `python common.py` ====================================
if __name__ == "__main__":
    set_seed()
    print("Loading train + val ...")
    train_ds, val_ds = get_train_val()
    print(f"  train: {len(train_ds)}   val: {len(val_ds)}")

    print("Loading test ...")
    test_ds = load_test()
```

```
    print(f"  test:  {len(test_ds)}")

    print("Building/loading answer vocab ...")
    vocab = get_answer_vocab()
    print(f"  |answer_vocab| = {len(vocab)}")

    print("\nFirst training example keys:", list(train_ds[0].keys()))
    print("Question:", train_ds[0]["question"])
    print("Choices: ", train_ds[0]["choices"])
    print("Answer:  ", train_ds[0]["answer"])
    print("Modality:", train_ds[0]["metadata"]["modality"])

    # Dummy evaluation: predict the most-frequent answer for everything.
    most_common_answer = Counter(train_ds["answer"]).most_common(1)[0][0]
    dummy_preds = [most_common_answer] * len(val_ds)
    print_eval(evaluate(dummy_preds, val_ds), title="Majority-class baseline (val)")
```